理工体剣道部　Web 開発ゼミ

剣道部サイトで
まなぶ Web 開発

第 1 巻　準備と、はじめての Web ページ

対象：プログラミング未経験・Windows

使うもの：Git / GitHub / VS Code / Claude Code / HTML / CSS / JavaScript

作成：三角 優斗　　2026 年 11 月版

目次

第 0 章　この教材の歩き方 ・・・

第 4 章　ターミナルに慣れる ・・・

7.8 スマホに対応する：レスポンシブ ・・・・・・・・・・・・・・・・・・・・・・・・・ 55
7.9 開発者ツール ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 56
7.10 GitHub Pages で公開する ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 57
7.11 練習問題 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 58
第 8 章 フェーズ①’ JavaScript で動きをつける・・・・・・・・・・・・・・・・・・・・・ 64
8.1 JavaScript を動かしてみる ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 64
8.2 変数とデータの型 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 65
8.3 配列とオブジェクト ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 65
8.4 関数・・・ 66
8.5 条件分岐と繰り返し ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 66
8.6 配列を変換する：filter と map ・・・・・・・・・・・・・・・・・・・・・・・・・ 67
8.7 DOM：JavaScript から HTML を操作する ・・・・・・・・・・・・・・・・・・・ 67
8.8 作ってみる・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 68
8.9 エラーの読み方・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 74
8.10 練習問題 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 75
8.11 第 1 巻のおわりに ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 79
付録 A コマンド早見表・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 80
付録 B エラー逆引き ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 82
付録 C 用語集 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 83
付録 D 練習問題の解答例 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 85

第 0 章　この教材の歩き方

0.1　ゴール

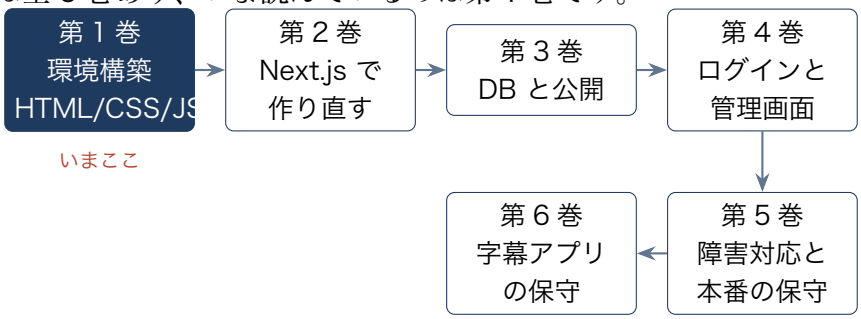
この教材のゴールは、**理工体剣道部の公式サイトを、自分で保守・運用できるようになること**です。保守というのは、たとえば次のようなことを、三角に頼らず自分で片づけられる状態を指します。

- 新しい年度の部員名簿や大会結果を、管理画面から反映できる
- 「ここの文言を直して」「写真を差し替えて」と頼まれたら、コードを直して公開できる
- 「サイトが表示されない」と言われたとき、どこが壊れているのかを絞り込める
- 字幕アプリの処理が失敗したとき、やり直してよいかどうかを判断できる

そのために、部のサイトの縮小版を、自分の手で一から作り直します。完成品はすでにあるので、作り直すことに意味があるのか不思議に思うかもしれません。けれども、**自分で一度組み立てたものは、壊れたときに直せる**ようになります。これがこの教材の一番の狙いです。

0.2　全体の地図

この教材は全 6 巻あり、いま読んでいるのは第 1 巻です。



第 1 巻では、開発に使う道具をそろえたあと、HTML・CSS・JavaScript だけで部のトップページを作ります。完成したら、インターネットに公開するところまで進めます。

? なぜいきなり Next.js から始めないのか

本番のサイトは Next.js というフレームワークで作られています。ただ、Next.js は「HTML・CSS・JavaScript を楽に書くための仕組み」なので、土台の 3 つを知らないまま使うと、エラーが出たときに何が起きているのか分かりません。第 1 巻で素の HTML・CSS・JavaScript を一度書いておくと、第 2 巻で Next.js を使ったときに「あの面倒な作業を、これが代わりにやってくれているのか」と納得しながら進められます。

0.3　教材と練習用リポジトリの場所

この教材と、第 2 巻から使う練習用のリポジトリは、GitHub の次の場所にあります。

```
https://github.com/yutomi141/rikotaikendo-practice
```

教材の PDF：`docs/textbook/vol1.pdf`（第 2 巻以降も同じ場所に増えていく）

- 第 1 巻のうちは、この PDF を読むだけで大丈夫です。リポジトリへの招待は第 2 章、手元へのコピー（clone）は第 6 章で行います。
- 教材が新しくなったら、三角が LINE で知らせます。最新の PDF は、いつでも上の場所から取れます。
- 教材の元のファイル（`.tex`）も同じ場所にあるので、第 6 章以降は Claude Code に「教材の第 7 章のここが分からない」と聞くこともできます。

0.4　この教材の読み方：3 つのルール

1. **コマンドは自分の手で打つ。**コピー＆ペーストでも動きますが、手で打つと「どこで区切るのか」「何を指定しているのか」が体に入ります。長いもの（URL や鍵）だけはコピーして構いません。
2. **分からない単語は、付録の用語集を引く。**それでも分からなければ、Claude に聞いてください（第 6 章で聞き方を説明します）。
3. **30 分詰まったら、人に聞く。**30 分考えて分からないことは、ひとりで 3 時間考えてもたいてい分かりません。聞き方の型は次の節で説明します。

0.5　進み具合の管理：スプレッドシート

やることは、すべて三角が用意した Google スプレッドシート（以下「スプシ」）に 1 行ずつ並んでいます。この教材のところどころに　スプシ 1-3 のような印があり、スプシの行と対応しています。

列	書くこと
ステータス	未着手 → 進行中 → レビュー待ち → 完了。手をつけたら「進行中」にする
着手日・完了日	その日の日付（Ctrl+; で今日の日付が入る）
PR リンク	PR を出したら URL を貼る（第 2 巻から）
詰まりメモ	詰まったことを下の「質問の型」で書く。解決したら解決方法も追記する
先輩確認	**三角が付けるので触らない**

シートの一番上には、フェーズごとの完了率と「必須の 5 項目」が表示されます。必須の 5 項目は、この教材全体を通して最終的にできるようになってほしいことです。

0.6　質問の型

人に質問するときも、Claude に聞くときも、次の 4 つをそろえます。

質問の型（4 点セット）

やりたいこと何をしようとしていたか（例：STEP 3 で `git push` しようとした）

期待したことどうなるはずだったか（例：GitHub にファイルが表示される）

実際　何が起きたか。**エラーメッセージは全文コピーして貼る**

試したこと　自分で試したこと、調べたこと、Claude の回答

「動きません」だけでは、聞かれた側は何も分かりません。4 点セットを書いている途中で自分で答えに気づくことも、意外とよくあります。

0.7　各フェーズの終わりにすること

各フェーズの最後には、次の 2 つがあります。

腕だめし手本どおりではなく、少しだけ違う課題です。**まず自分で「どのファイルの、どこを、どう変えれば、どうなるはずか」をメモしてから**、手を動かします（Claude を使ってもかまいません）。

説明タスク作ったものの仕組みを、三角に自分の言葉で説明します。定例の時間を使います。うまく説明できなかったところは宿題にして、次の定例で説明し直します。

0.8　AI と一緒に作るということ

この教材は、Claude Code（AI）に手伝ってもらいながら進める前提で書いてあります。AI に書かせること自体は問題ありません。ただし、次の線だけは守ってください。

⚠ 完了と言ってよい条件

1. **何を変えたか、なぜそう変えたかを、自分の言葉で説明できる**
2. **動作を自分の目で確かめた**（どう確かめたかを書ける）
3. **差分（変更前と変更後の違い）を自分で読んだ**

この 3 つがそろわない状態の「動いたから OK」は、完了ではありません。本番のサイトを保守するようになったとき、説明できないコードは直せないからです。

0.9　予定の目安

期限はありません。週に 5〜8 時間ほど使えると、第 1 巻は 1 か月前後で終わる分量です。定例は週 1 回、15〜30 分です。進み具合の確認と、説明タスクに使います。

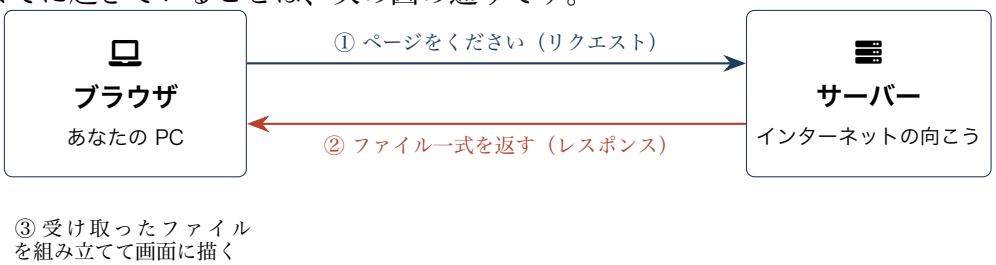
第 1 章　Web の世界地図

スプシ 1-1〜1-3

道具をそろえる前に、これから作るものの全体像を見ておきます。細かい言葉は覚えなくて大丈夫です。この章は、迷ったときに戻ってくる「地図」として使ってください。

1.1　Web ページが表示されるまで

ブラウザに `https://rikotaikendo.jimdofree.com` と打ち込んでから、ページが表示されるまでに起きていることは、次の図の通りです。



- **ブラウザ**：Chrome や Edge のこと。受け取ったファイルを読んで画面を描きます。
- **サーバー**：ページの元になるファイルを置いておき、頼まれたら返すコンピュータです。
- **リクエストとレスポンス**：「ください」と「どうぞ」のやりとりです。Web の通信はすべてこれの繰り返しです。

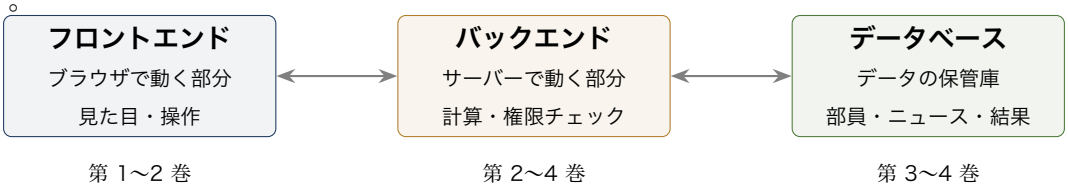
ブラウザが受け取るファイルは、主に次の 3 種類です。

ファイル	役割	たとえると
HTML	何が書いてあるか（文章・見出し・画像の場所）	家の骨組み
CSS	どう見せるか（色・大きさ・配置）	内装・ペンキ
JavaScript	どう動くか（クリックしたら開く、絞り込む）	電気・スイッチ

第 1 巻の第 7 章で HTML と CSS を、第 8 章で JavaScript を書きます。

1.2　フロントエンド・バックエンド・データベース

部のサイトには、ニュースや大会結果を部員が更新できる「管理画面」があります。更新された内容はどこかに保存しておかなければなりません。保存先が**データベース**（DB）です。

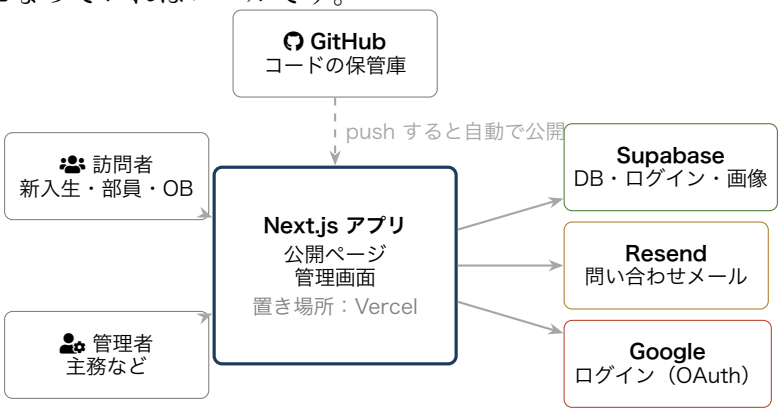


なぜバックエンドを挟むのか

ブラウザから直接データベースを書き換えられると、誰でも部員名簿を消せてしまいます。そこで「この人は管理者か？」を確かめる係が必要になります。それがバックエンドの大事な仕事です。第４巻で、この「確かめる係」を自分で作ります。

1.3　部のサイトの構成

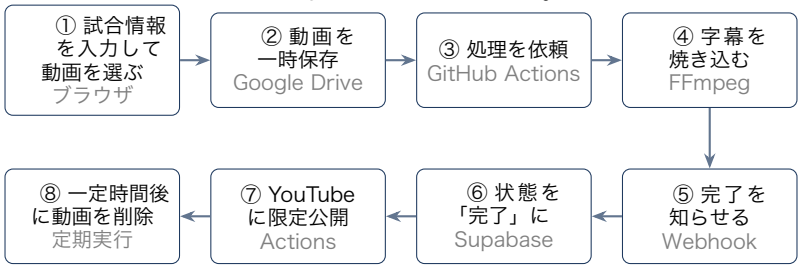
本番の `rikotaikendo-web` は、次のような部品の組み合わせでできています。すべてを今理解する必要はありません。第 6 巻を終えたとき、この図のどの部品についても説明できるようになっていればゴールです。



部品	何をしているか	学ぶ巻
GitHub	コードを保管し、変更の履歴を残す。チームでの共同作業の場	第 1 巻〜
Next.js	ページを作る仕組み（フレームワーク）	第 2 巻
Vercel	Next.js のアプリをインターネットに公開する場所	第 3 巻
Supabase	データベース、ログイン、画像の保存をまとめて提供するサービス	第 3〜4 巻
Google OAuth	「Google でログイン」の仕組み	第 4 巻
Resend	問い合わせフォームからメールを送るサービス	第 5 巻

1.4　字幕アプリの構成

すでに運用で使っている字幕アプリも、ほとんど同じ部品でできています。違うのは、**重い動画処理を GitHub Actions に任せている**点です。



10 月の大会では、外部サーバーの不調で 3 本の処理が失敗しました。あれは、この図の③〜④のどこかで止まっていたということです。第 6 巻では、こうした失敗が「どこで」起きたのかを自分で突き止められるようにします。

STEP 1　三角と一緒に、字幕アプリの裏側をたどる（定例で実施）

スプシ 1-3 定例の時間を 1 回使い、三角の画面共有で、試合動画 1 本の処理を最初から最後まで追いかけます。見ながら、下の表の空欄を埋めてください。

上の図の番号	どの画面で確認できたか／何が表示されていたか
① 入力	
③ Actions の実行	
④ FFmpeg のログ	
⑥ DB の状態	
⑦ YouTube	

1.5　ローカルとクラウド

最後に、言葉を 2 つだけ覚えておきます。

- **ローカル**：自分の PC の中のこと。作業中のファイルはまずここにあります。
- **クラウド**（リモート）：インターネットの向こうにあるコンピュータのこと。GitHub や Vercel がこれにあたります。

開発は「ローカルで作って試す → クラウドに送って公開する」の繰り返しです。ローカルから GitHub に送る操作を **push**（プッシュ）と呼びます。第 5 章で実際にやります。

☑ できるようになったかチェック

- □ブラウザとサーバーのやりとりを、リクエスト・レスポンスという言葉を使って説明できる
- □HTML・CSS・JavaScript の役割の違いを言える
- □管理画面とデータベースの間に、なぜバックエンドが必要なのかを言える
- □字幕アプリの処理が、どの順番でどのサービスを通るのかを図で示せる

第 2 章　アカウントをそろえる

スプシ 2-1〜2-3

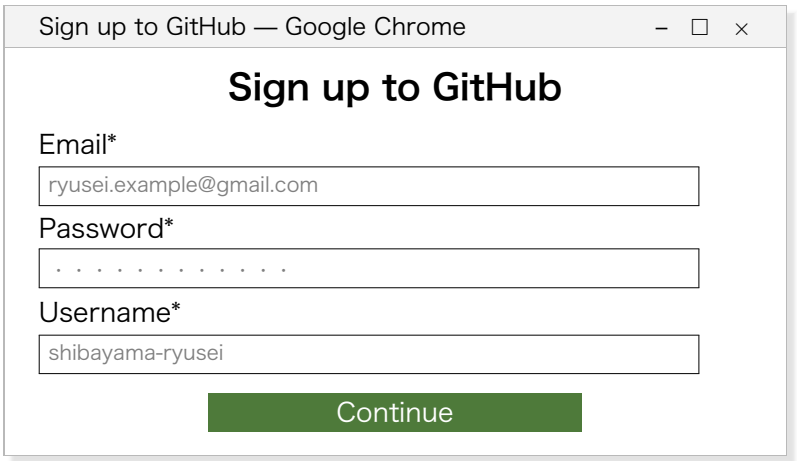
開発で使うサービスのアカウントを用意します。第 1 巻で必要なのは、GitHub と Claude の 2 つだけです。Vercel と Supabase は、使う巻（第 3 巻）になったら作ります。

2.1　GitHub

GitHub（ギットハブ）は、コードを保管し、変更の履歴を残し、他の人と共同で作業するためのサービスです。本番のサイトのコードも、字幕アプリのコードも GitHub にあります。

STEP 1　GitHub のアカウントを作る

1. ブラウザで `https://github.com/signup` を開きます。
2. メールアドレス、パスワード、ユーザー名を入力します。
3. 届いたメールの確認コードを入力すると、登録が完了します。



GitHub の登録画面（模式図）

⚠ ユーザー名の決め方

ユーザー名は、URL（`github.com/ユーザー名`）にも、コミットの記録にも残ります。あとから変えると面倒なので、次の点に気をつけて決めてください。

- 英小文字・数字・ハイフンだけで作る（例：`shibayama-ryusei`、`ryusei-s`）
- 就職活動などで人に見せても恥ずかしくない名前にする

STEP 2　2 段階認証をオンにする

GitHub は 2 段階認証が必須です。登録後しばらくすると設定を求められるので、スマホの認証アプリ（Google Authenticator や Microsoft Authenticator など）で設定します。**リカバリーコードは必ず保存してください。**スマホをなくしたときに、これがないとアカウントに入れなくなります。

STEP 3　学生向け特典（GitHub Education）を申請する

`https://education.github.com/` から、**keio.jp のメールアドレス**で学生認証を申請すると、有料機能の一部が無料になります。必須ではありませんが、通っておくと後で役立ちます。審査には数日かかることがあります。

STEP 4　ユーザー名を三角に伝える

スプシ 2-2 練習用のリポジトリにあなたを招待するため、GitHub のユーザー名を LINE で三角に送ります。

STEP 5　練習用リポジトリへの招待を承認する

スプシ 2-2 三角が招待すると、GitHub に登録したメールアドレスに「yutomi141 invited you to yutomi141/rikotaikendo-practice」というメールが届きます。メールの View invitation を押し、開いた画面で Accept invitation を押します。メールが見つからないときは、`https://github.com/yutomi141/rikotaikendo-practice/invitations` を直接開いても承認できます。

承認すると、このリポジトリにブランチを作ったり、PR を出したりできるようになります（第 2 巻から使います）。

⚠ 招待には期限がある

GitHub の招待は、7 日たつと無効になります。期限が切れたら、三角にもう一度招待してもらってください。

2.2 Claude

Claude Pro にはすでに入っているので、ここでは 2 つの点を確認するだけです。

STEP 6　プランと使用量を確認する

1. https://claude.ai にログインし、左下のアカウント名から **設定** を開きます。
2. **使用量**（Usage）の画面で、いまの使用量を確認します。

Pro の使用量の仕組み

- 使用量には「5 時間ごとの上限」と「週ごとの上限」があります。
- **ブラウザやスマホで Claude とチャットした分と、Claude Code で使った分は、同じ枠から引かれます。**開発を進めたい日は、チャットでの雑談を控えめにすると長持ちします。
- 上限に達したら、リセットされるまで待ちます。待っている間にできることは第 6 章で紹介します。

追加の課金は押さない

上限に達すると、追加のクレジットを買うかどうか聞かれることがあります。**この教材では課金は一切不要です。**「購入」「有効にする」は押さず、リセットを待ってください。

できるようになったかチェック

- □GitHub のアカウントを作り、2 段階認証を設定した
- □リカバリーコードを安全な場所に保存した
- □GitHub のユーザー名を三角に伝えた
- □Claude の使用量の画面を開けた

第 3 章　Windows の開発環境をつくる

スプシ 3-1〜3-9

この章では、開発に使う道具を PC にインストールします。この教材全体で一番つまずきやすいところなので、**STEP の順番を守り、各 STEP の「確認」まで終えてから次へ進んでください。**

順番	道具	何に使うか
1	VS Code	コードを書くエディタ（Word のコード版）
2	Git for Windows	変更の履歴を管理する Git と、黒い画面の Git Bash
3	Git の初期設定	名前・メールアドレスの登録
4	SSH 鍵	パスワードなしで GitHub とやりとりするための鍵
5	Node.js	JavaScript を PC 上で動かす。第 2 巻以降で必須
6	Claude Code	AI に開発を手伝ってもらう

? なぜ VS Code を先に入れるのか

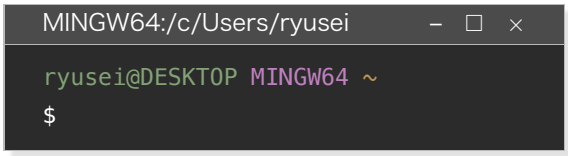
Git のインストール中に「Git が使うエディタはどれにするか」を聞かれます。そのとき VS Code が入っていれば選択肢に出てくるので、先に入れておきます。

3.1　2 種類の「黒い画面」：PowerShell と Git Bash

Windows には、文字でコンピュータに命令する画面（**ターミナル**）が何種類かあります。この教材で使うのは次の 2 つです。



PowerShell
Windows に最初から入っている。
インストール作業のときだけ使う。



Git Bash
Git と一緒に入る。
普段の作業はすべてこちらで行う。

? なぜ Git Bash を使うのか

本番のサイトを動かしている Vercel のサーバーも、三角の Mac も、中身は同じ系統の仕組み（Unix 系）で動いています。Git Bash はその Unix 系と同じ命令が使えるので、ネットで見つけた手順や Claude が出すコマンドがそのまま通ります。大学の

Git 演習の授業で使っているのも Git Bash です。

見分け方は簡単で、**行の先頭が `PS` なら PowerShell、`$` なら Git Bash** です。この教材の黒い枠には、左上にどちらで打つのかが書いてあります。

3.2　作業フォルダの場所を決める

⚠ 作業フォルダを置いてはいけない場所

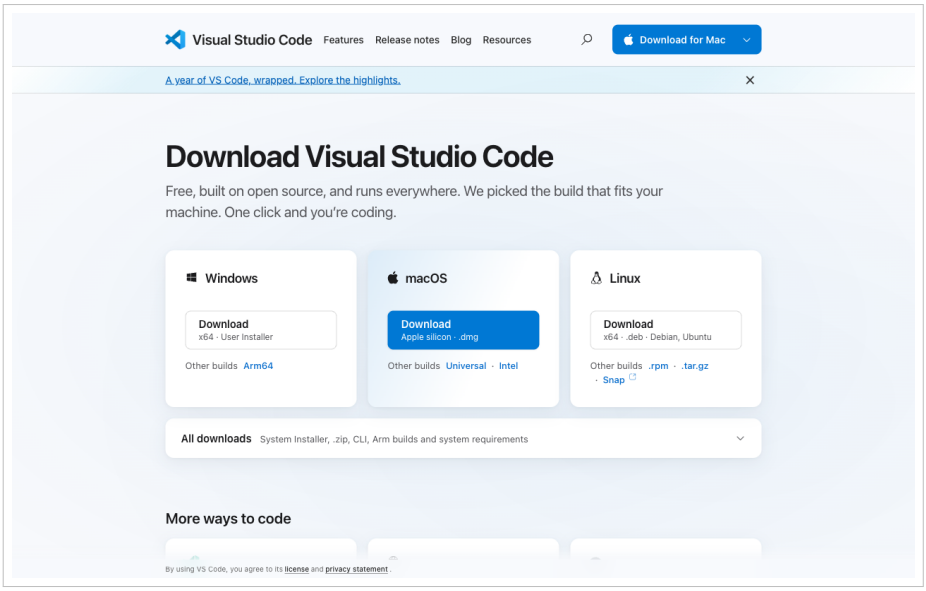
- **デスクトップ、ドキュメント**：OneDrive と同期していることが多く、同期の途中でファイルが壊れたり、`node_modules`（第 2 巻で出てくる大量のファイル）の同期で PC が重くなったりします。
- **日本語やスペースを含むフォルダ**：一部の道具がうまく動きません。

この教材では、作業フォルダを `C:\Users\(あなたのユーザー名)\dev` に統一します。Git Bash では、これを `~/dev` と書きます（~ はホームフォルダを表す記号で、「チルダ」と読みます）。フォルダ自体は、第 4 章で自分のコマンドを使って作ります。

3.3　VS Code

STEP 1　VS Code をダウンロードしてインストールする

1. `https://code.visualstudio.com/download` を開き、**Windows の Download**（x64 User Installer）を押します。



2. ダウンロードした `VSCodeUserSetup-x64-(バージョン).exe` を開きます。
3. 「追加タスクの選択」の画面では、**次の項目すべてにチェックを入れます。**



「PATH への追加」は特に重要。ターミナルから `code` コマンドで VS Code を開けるようになる

4. インストール を押し、終わったら 完了 を押します。

STEP 2　日本語化と、改行コードの設定

1. VS Code の左端にある四角が 4 つのアイコン（拡張機能）を押し、`Japanese Language Pack` を検索してインストールします。右下に出る Restart を押すと、メニューが日本語になります。
2. Ctrl+, で設定を開き、上の検索欄に `eol` と打ちます。
3. **Files: Eol** を `\n` に変更します。

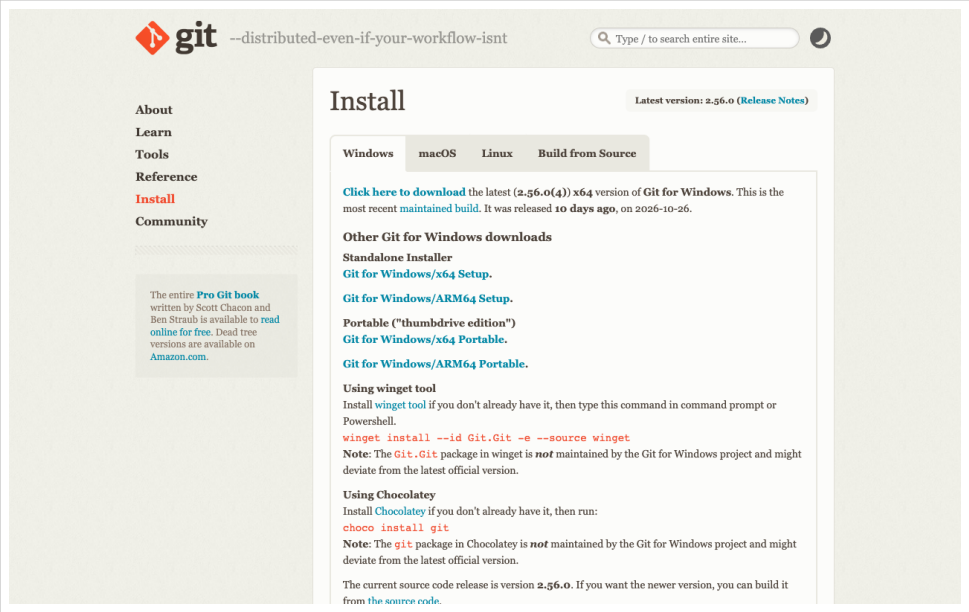
⑦ 改行コードとは

文章の「改行」は、コンピュータの中では見えない記号で表されています。Windows は伝統的に `CRLF`、Mac や Linux やサーバーは `LF` という別の記号を使います。混ざると、中身は変わっていないのに Git が「全行変更された」と判断してしまいます。そこで、この教材では `LF`（`\n`）にそろえます。

3.4　Git for Windows

STEP 3　Git をダウンロードする

`https://git-scm.com/downloads/win` を開き、一番上の **Click here to download** を押します。



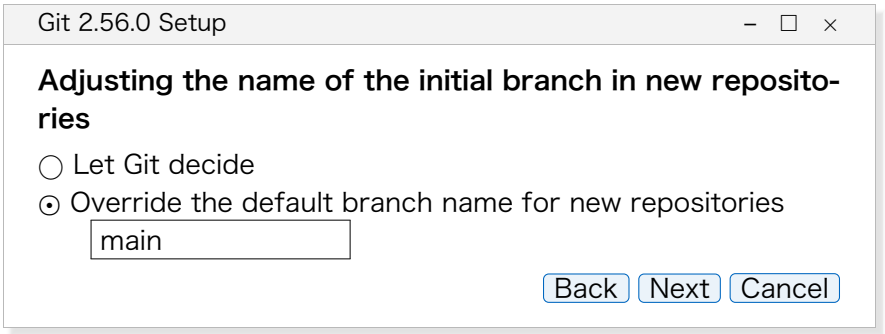
2026 年 11 月時点の最新版は 2.56.0。数字が違っていても、一番上のリンクで問題ない

STEP 4　インストーラーを進める

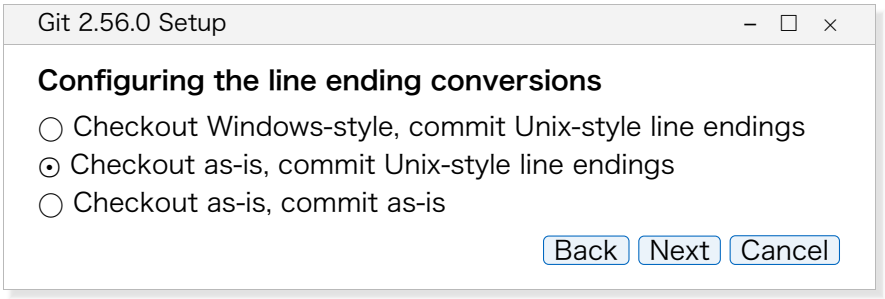
ダウンロードした `Git-(バージョン)-64-bit.exe` を開きます。画面がたくさん出てきますが、**変更するのは 3 か所だけ**です。それ以外は何も変えずに Next を押してください。



変更①：エディタを Vim から **Visual Studio Code** に変える（Vim は操作が難しく、初心者が抜け出せなくなる）



変更②：下を選び、`main` と入力する（GitHub の標準に合わせる）



変更③：**真ん中**を選ぶ（改行コードを LF にそろえるため）

最後に Install を押し、終わったら Finish を押します。

PATH の画面について

途中で「Adjusting your PATH environment」という画面が出ます。真ん中の **Git from the command line and also from 3rd-party software (Recommended)** が選ばれていることを確認してください（最初から選ばれているはずです）。

STEP 5　確認：Git Bash を開いてバージョンを見る

スタートメニューで「Git Bash」と検索して開き、次のように打ちます（`$` は打たなくてよい印です）。

```
Git Bash
$ git --version
git version 2.56.0.windows.1
```

`git version` の後に数字が出れば成功です。数字は違っていてもかまいません。

3.5　Git の初期設定

STEP 6　名前とメールアドレスを登録する

Git は、誰が変更したのかを記録します。そのための名前とメールアドレスを登録します。

Git Bash

```
$ git config --global user.name "shibayama-ryusei"
$ git config --global user.email "GitHub に登録したメールアドレス"
```

STEP 7　その他の設定と確認

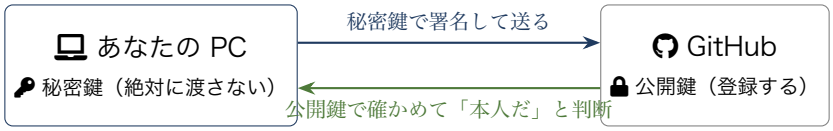
Git Bash

```
$ git config --global core.quotepath false
$ git config --global init.defaultBranch main
$ git config --global --list
user.name=shibayama-ryusei
user.email=....
core.quotepath=false
init.defaultbranch=main
core.autocrlf=input
```

`core.quotepath false` は、日本語のファイル名が `\346\227...` のような記号の羅列で表示されるのを防ぐ設定です。最後の `core.autocrlf=input` は、インストーラーで選んだ改行コードの設定です。

3.6　SSH 鍵

GitHub にコードを送るとき、毎回パスワードを入力する代わりに「鍵」を使って本人確認をします。大学の Git 演習 第 3 回で扱った **公開鍵認証** と同じ仕組みです。



STEP 8　鍵を作る

Git Bash

```
$ ssh-keygen -t ed25519 -C "GitHub に登録したメールアドレス"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/ryusei/.ssh/id_ed25519):   ← Ent
Enter passphrase (empty for no passphrase):                              ← Ente
Enter same passphrase again:                                             ← Ente
```

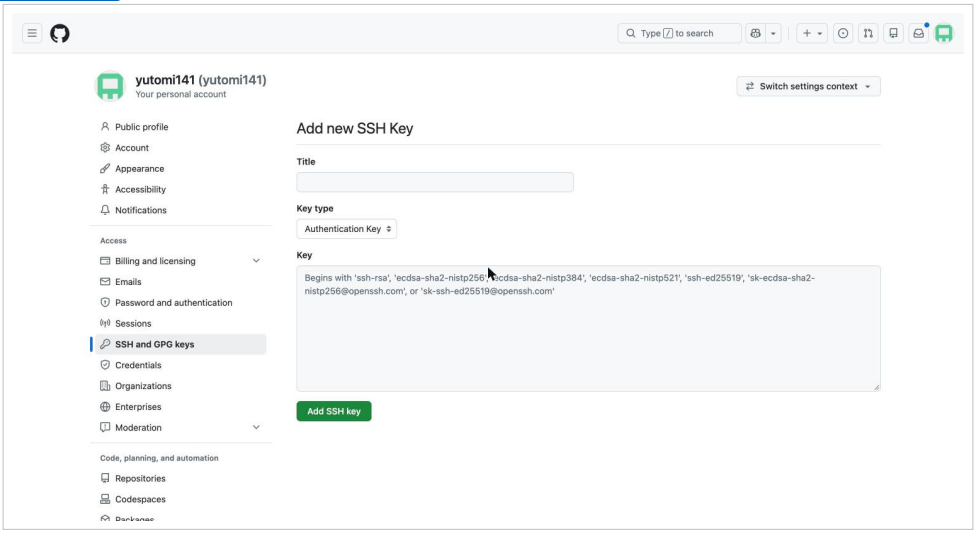
質問には 3 回とも何も打たずに `Enter` を押します。

STEP 9　公開鍵をコピーして GitHub に登録する

Git Bash

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

これで公開鍵がクリップボードにコピーされます（`clip` は Windows のコピー用コマンドです）。GitHub で `https://github.com/settings/ssh/new` を開き、**Title** に `my-windows-pc` のような名前を入れ、**Key** の欄に `Ctrl`+`V` で貼り付けて `Add SSH key` を押します。



⚠ 秘密鍵は絶対に見せない・送らない

`~/.ssh/id_ed25519`（`.pub` が付いていない方）は秘密鍵です。これが他人に渡ると、あなたになりすまして GitHub を操作できてしまいます。Claude に見せるのも、LINE で送るのも禁止です。登録するのは `.pub` が付いた公開鍵だけです。

STEP 10　確認：GitHub につながるか試す

Git Bash

```
$ ssh -T git@github.com
The authenticity of host 'github.com (...)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Hi shibayama-ryusei! You've successfully authenticated, but GitHub does not pro
```

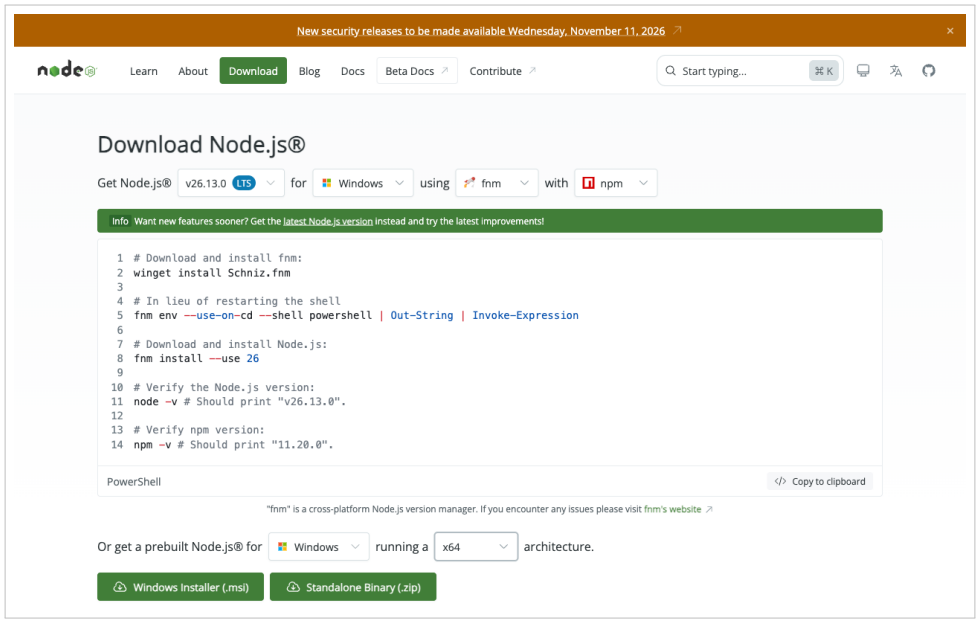
初回だけ確認を聞かれるので `yes` と打ちます。`Hi (ユーザー名)!` と出れば成功です。

3.7　Node.js

Node.js（ノード）は、ブラウザの外で JavaScript を動かすための道具です。第 1 巻では、作ったページを PC 上で表示するとき（第 7 章）に使います。第 2 巻以降は Next.js を動かすのに必須です。

STEP 11　Node.js をインストールする

1. `https://nodejs.org/en/download` を開きます。
2. 下の方の「Or get a prebuilt Node.js for」で、**Windows** と **x64** を選び、Windows Installer (.msi) を押します。



バージョンは **LTS**（長期サポート版）を選ぶ。2026 年 11 月時点では v26

3. ダウンロードした `.msi` を開き、すべて Next で進めます。
4.「Tools for Native Modules」の画面では、**チェックを入れずに**進めます。

STEP 12　確認：Git Bash を開き直してバージョンを見る

インストール前から開いていた Git Bash は、一度閉じて開き直してください（新しく入った道具は、開き直さないと見つかりません）。

Git Bash
```
$ node -v
v26.13.0
$ npm -v
11.20.0
```

PowerShell で npm を打つとエラーになる場合

PowerShell で `npm` を打つと「このシステムではスクリプトの実行が無効になっている」というエラーが出ることがあります。これは PowerShell の安全機能によるものです。この教材では **npm は Git Bash で打つ**ので、気にしなくて大丈夫です。

3.8　Claude Code

STEP 13　PowerShell でインストールする

Claude Code のインストールだけは **PowerShell** で行います。スタートメニューで「PowerShell」と検索して開き（管理者として実行する必要はありません）、次のように打ちます。

PowerShell
```
PS C:\Users\ryusei> irm https://claude.ai/install.ps1 | iex
```

しばらく何も表示されないまま待たされますが、正常です。終わるとプロンプト（`PS C:\...>`）に戻ります。

STEP 14　確認：Git Bash で動くか試す

Git Bash を**新しく開き直して**、次のように打ちます。

Git Bash

```
$ claude --version
2.x.xxx (Claude Code)
$ claude doctor
```

バージョンが表示されれば成功です。`claude doctor` は設定の健康診断です。警告が出たら内容を控えておき、スプシの詰まりメモに書いてください。

claude: command not found と出たら

インストール先のフォルダ（`C:\Users\(ユーザー名)\.local\bin`）が、コマンドを探す場所（PATH）に入っていない状態です。次の手順で追加します。

1. スタートメニューで「環境変数」と検索し、**環境変数を編集**（ユーザー用）を開く
2. 上の欄の **Path** を選んで 編集 を押す
3. 新規 を押して `%USERPROFILE%\.local\bin` と入力し、OK で閉じる
4. Git Bash をすべて閉じてから開き直す

STEP 15　ログインする

Git Bash

```
$ claude
```

初回はブラウザが開くので、**Claude Pro のアカウント**でログインします。ログイン方法を聞かれたら、**Claude アカウント（サブスクリプション）**の方を選んでください。ログインが終わったら、`/exit` と打って一度終了します。

API キーは設定しない

Claude Code は、`ANTHROPIC_API_KEY` という設定があると、Pro の定額の枠ではなく**使った分だけ課金される API** の方を使います。ネットの記事で「API キーを設定しよう」と書いてあっても、**この教材では設定しません**。念のため、次のコマンドで何も表示されないことを確認しておきます。

Git Bash

```
$ echo $ANTHROPIC_API_KEY
```

（空行が出るだけなら OK です）

3.9　VS Code のターミナルを Git Bash にする

ここまでは Git Bash を単独で開いてきましたが、普段は VS Code の中にあるターミナルを使うと便利です。

STEP 16　VS Code の標準ターミナルを Git Bash にする

1. VS Code で `Ctrl`+`Shift`+`P` を押し、`Terminal: Select Default Profile` と打って選びます。
2. 一覧から **Git Bash** を選びます。
3. `Ctrl`+`@` でターミナルを開き、`$` で始まる行が出れば成功です。

☑　環境構築の最終チェック（Git Bash ですべて打つ）

□`git --version` でバージョンが出る

□`git config --global user.name` で自分のユーザー名が出る

□`ssh -T git@github.com` で `Hi (ユーザー名)!` と出る

□`node -v` と `npm -v` でバージョンが出る

□`claude --version` でバージョンが出る

□`echo $ANTHROPIC_API_KEY` で何も出ない

□VS Code で `Ctrl`+`@` を押すと Git Bash が開く

ここまで終わったら、スプシの 3-1〜3-9 を「完了」にしてください。

第 4 章　ターミナルに慣れる

スプシ 4-1〜4-3

この章では、Git Bash でフォルダやファイルを操作する練習をします。エクスプローラーでマウスを使ってやっていることを、文字で命令するだけです。

4.1　GUI と CLI

	GUI（グラフィカル）	CLI（コマンドライン）
操作	マウスでアイコンをクリック	文字で命令を打つ
例	エクスプローラー、Word	Git Bash、PowerShell
得意なこと	見て分かる、迷わない	繰り返し・正確さ・記録に残る・AI と相性がよい

開発で CLI を使う最大の理由は、**やったことが文字で残る**ことです。「さっき何をしたっけ？」と聞かれたときに、打ったコマンドをそのまま見せれば済みます。Claude Code が裏で実行しているのも、同じコマンドです。

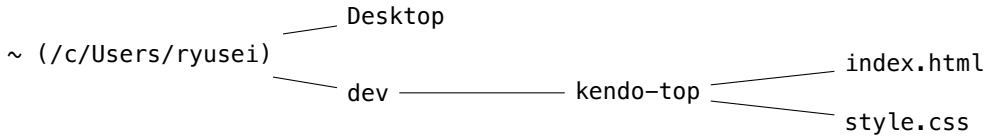
4.2　プロンプトの読み方



- `ryusei@DESKTOP-AB12CD`：ユーザー名@PC の名前
- `~/dev`：**いま居る場所**（カレントディレクトリ）。`~` はホームフォルダ `C:\Users\ryusei` のこと
- `$`：ここに命令を打ってください、という印

4.3　パス：ファイルの住所

パスはファイルやフォルダの「住所」です。Git Bash では区切りに `/`（スラッシュ）を使います。



- **絶対パス**：一番上から書く住所。例：`/c/Users/ryusei/dev/kendo-top/index.html`
- **相対パス**：いま居る場所から見た住所。`dev` に居るなら `kendo-top/index.html`
- `.`（ドット 1 つ）は「いま居る場所」、`..`（ドット 2 つ）は「1 つ上の階層」

4.4　基本コマンド

コマンド	由来	すること
`pwd`	print working directory	いま居る場所を表示する
`ls`	list	中にあるファイル・フォルダを一覧表示する（`ls -a` で隠しファイルも）
`cd 場所`	change directory	移動する。`cd ..` で 1 つ上、`cd` だけでホームに戻る
`mkdir 名前`	make directory	フォルダを作る
`touch 名前`		空のファイルを作る
`cat ファイル`	concatenate	ファイルの中身を表示する
`cp 元 先`	copy	コピーする
`mv 元 先`	move	移動する／名前を変える
`rm ファイル`	remove	削除する（**ゴミ箱に入らない**）
`code .`		いまの場所を VS Code で開く
`explorer .`		いまの場所をエクスプローラーで開く
`clear`		画面をきれいにする（Ctrl+L でも同じ）

⚠ rm は取り消せない

`rm` で消したファイルはゴミ箱に入らず、そのまま消えます。特に `rm -rf`（フォルダごと強制削除）は、場所を間違えると大事なものまで消えます。Claude Code が `rm -rf` を実行しようとしたら、**どこを消そうとしているのか必ず確かめてから**許可してください。

4.5　便利な操作

操作	効果
Tab	名前の続きを自動で補完する。**打ち間違いが減るので積極的に使う**
↑ ↓	前に打ったコマンドを呼び出す
Ctrl+C	実行中のコマンドを止める（Git Bash ではコピーではない）
Shift+Insert	貼り付け（右クリック → Paste でも可）
q	`git log` などの一覧表示から抜ける

4.6　ハンズオン

STEP 1　作業フォルダを作る

スプシ 4-2

Git Bash

```
$ cd
$ pwd
/c/Users/ryusei
$ mkdir dev
$ cd dev
$ pwd
/c/Users/ryusei/dev
```

これで、作業フォルダ `~/dev` ができました。この先の作業は、すべてこの中で行います。

STEP 2　練習用のフォルダとファイルを作って動かす

Git Bash

```
$ mkdir terminal-practice
$ cd terminal-practice
$ touch memo.txt
$ echo "面打ち 100 本" > memo.txt
$ cat memo.txt
面打ち 100 本
$ echo "小手面 50 本" >> memo.txt
$ cat memo.txt
面打ち 100 本
小手面 50 本
$ cp memo.txt memo-backup.txt
$ ls
memo-backup.txt  memo.txt
$ mv memo-backup.txt old.txt
$ ls
memo.txt  old.txt
$ rm old.txt
$ ls
memo.txt
```

`>` はファイルに上書き、`>>` はファイルの末尾に追記、という意味です。

STEP 3　VS Code とエクスプローラーで確かめる

Git Bash

```
$ code .
$ explorer .
```

VS Code とエクスプローラーの両方で `memo.txt` が見えれば、「ターミナルで操作したものと、マウスで見ているものは同じもの」だと確かめられたことになります。

腕だめし：ターミナルだけで部の稽古メモを作る

スプシ 4-3 マウスを使わずに、次の状態を作ってください。

1. `~/dev/terminal-practice` の中に `keiko` というフォルダを作る
2. その中に `monday.txt` と `thursday.txt` を作り、それぞれに稽古メニューを 2 行ずつ書く

3. `thursday.txt` を `saturday.txt` という名前に変える
4. `cat` で 2 つのファイルの中身を表示し、その画面をスクショしてスプシの詰まりメモ欄に「完了」と書く

☑ できるようになったかチェック

- □プロンプトを見て、いま居る場所を言える
- □絶対パスと相対パスの違いを説明できる
- □`cd`・`ls`・`mkdir`・`cat` を、表を見ずに使える
- □`Tab` で補完できる
- □`rm` が危ない理由を言える

第 5 章　Git と GitHub の基本

スプシ 5-1〜5-6

5.1　Git がないと何が困るか

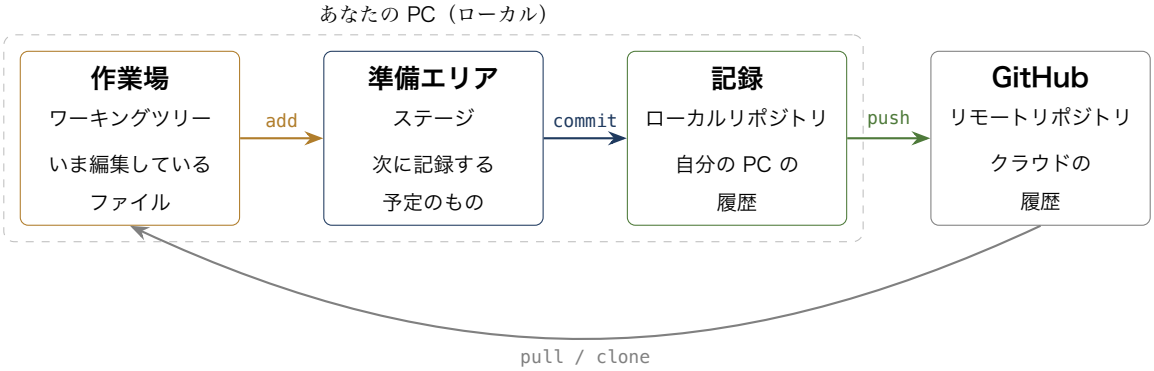
レポートを書いていて、こんなファイルが並んだことはありませんか。

```
レポート.docx
レポート_修正.docx
レポート_修正 2.docx
レポート_最終版.docx
レポート_最終版_v2_本当に最終.docx
```

どれが最新なのか、何を変えたのか、どこで壊れたのかが分からなくなります。これを人数分・ファイル数百個分でやるのが Web 開発です。**Git**（ギット）は、ファイルを上書きしながら、**いつ・誰が・何を・なぜ変えたか**を全部記録してくれる道具です。

5.2　Git の 3 つの場所

Git を使うとき、ファイルは次の 4 つの場所を行き来します。最初はこの図だけ覚えれば十分です。



add「このファイルの変更を次の記録に入れる」と選ぶ

commit選んだ変更を、メッセージ付きで 1 つの記録（**コミット**）にする。ゲームのセーブに近い

push自分の PC にある記録を GitHub に送る

pull / clone GitHub にある記録を自分の PC に持ってくる

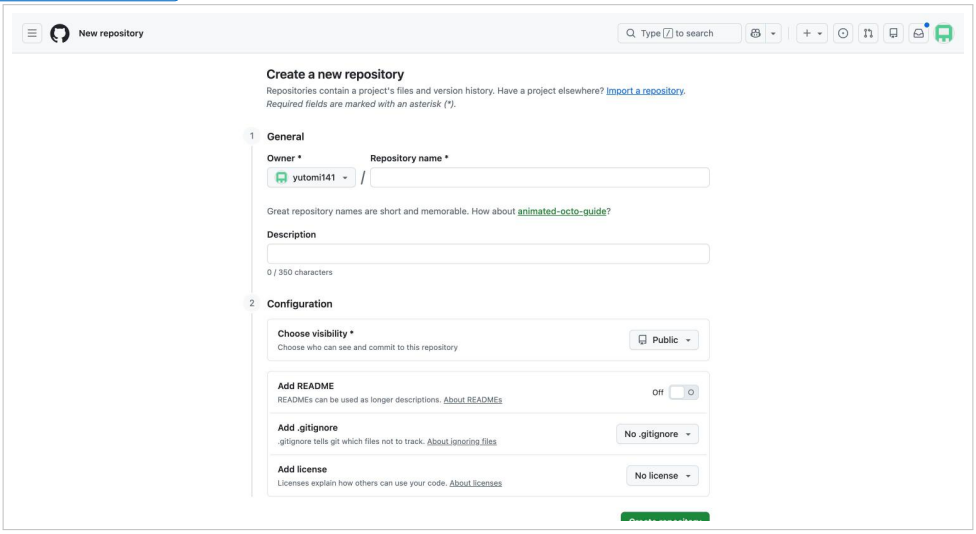
? なぜ add と commit が分かれているのか

1 回の作業で「トップページの文言修正」と「FAQ の追加」を同時にやってしまったとします。add で文言修正のファイルだけを選んで commit し、次に FAQ のファイルを選んで commit すれば、**1 つのコミットに 1 つの意味**を持たせられます。あとで「FAQ の追加だけ取り消したい」となったときに、これが効いてきます。

5.3 はじめてのリポジトリ

STEP 1 GitHub にリポジトリを作る

スプシ 5-2 `https://github.com/new` を開き、次のように入力して Create repository を押します。



- **Repository name**：`git-practice`
- **Choose visibility**：**Public**
- **Add README**：**Off のまま**（README は自分で作るため）

作ると、`…or push an existing repository from the command line` という説明が出た画面になります。この画面は開いたままにしておきます。

STEP 2　ローカルにリポジトリを作る

Git Bash

```
$ cd ~/dev
$ mkdir git-practice
$ cd git-practice
$ git init
Initialized empty Git repository in C:/Users/ryusei/dev/git-practice/.git/
$ ls -a
.  ..  .git
```

`.git` という隠しフォルダが、Git の記録帳です。**このフォルダは手で触らないでください。**

STEP 3　README を書く

Git Bash

```
$ code README.md
```

VS Code が開くので、次の内容を書いて Ctrl+S で保存します。

README.md

```
# git-practice

Git の練習用リポジトリです。

## 自己紹介

- 名前：柴山りゅうせい
- 所属：理工学部 応用化学科 2年／理工体剣道部
- 好きな技：（ここに書く）
```

`.md` は **Markdown**（マークダウン）という書式です。`#` が見出し、`-` が箇条書きになります。GitHub ではきれいに整えて表示されます。

STEP 4　status → add → commit

```
>_ Git Bash
$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

$ git add README.md
$ git status
Changes to be committed:
        new file:   README.md

$ git commit -m "docs: README を追加"
[main (root-commit) 3f2a1c9] docs: README を追加
 1 file changed, 9 insertions(+)
```

`git status` は何度打ってもかまいません。迷ったらまず `git status` です。いまどの場所に何があるのかを教えてくれます。

STEP 5　GitHub に push する

さっき開いておいた GitHub の画面にある **SSH** のアドレス（`git@github.com:...`）を使います。

```
>_ Git Bash
$ git remote add origin git@github.com:shibayama-ryusei/git-practice.git
$ git push -u origin main
...
To github.com:shibayama-ryusei/git-practice.git
 * [new branch]      main -> main
```

GitHub の画面を再読み込みして、README が表示されていれば成功です。`origin` は「GitHub 側のリポジトリ」につけたあだ名です。`-u` は初回だけ付ければよく、2回目からは `git push` だけで送れます。

5.4　コミットメッセージの書き方

この教材と本番のリポジトリでは、コミットメッセージを次の形で書きます（Conventional Commits）。

種類	使うとき	例
`feat`	新しい機能やページを足した	`feat: FAQ ページを追加`
`fix`	バグを直した	`fix: スマホで見出しがはみ出るのを修正`
`style`	見た目だけを変えた	`style: ボタンの色を紺に変更`
`docs`	説明文や README を変えた	`docs: README に起動方法を追記`
`refactor`	動きは同じまま、書き方を整理した	`refactor: 部員表示を関数に切り出す`
`chore`	設定などの雑務	`chore: .gitignore を追加`

よいコミットの大きさ

「1 つのコミット＝説明が 1 行で済む変更」が目安です。メッセージに「と」を書きたくなったら（「見出しを直したのと、FAQ を足した」）、コミットを分けるサインです。こまめにコミットしておくと、壊したときに戻れる地点が増えます。

5.5　差分を見る

STEP 6　変更を加えて diff を見る

README の「好きな技」を書き換えて保存してから、次を打ちます。

Git Bash

```
$ git diff
diff --git a/README.md b/README.md
@@ -7,4 +7,4 @@
 - 名前：柴山りゅうせい
 - 所属：理工学部 応用化学科 2 年／理工体剣道部
-- 好きな技：（ここに書く）
+- 好きな技：出小手
```

`-` で始まる行が消えた行、`+` で始まる行が足された行です。**`git diff` を読む力は、AI と開発するときに一番大事な力です。**Claude Code が何を変えたのかは、最終的にはこれで確かめます。

STEP 7　履歴を見る

変更を add・commit・push してから、履歴を見ます。

```
Git Bash
$ git add README.md
$ git commit -m "docs: 好きな技を追記"
$ git push
$ git log --oneline
a81d0e4 (HEAD -> main, origin/main) docs: 好きな技を追記
3f2a1c9 docs: README を追加
```

VS Code でも差分は見られる

VS Code の左端にある、枝分かれしたアイコン（ソース管理）を押すと、変更したファイルの一覧が出ます。ファイル名を押すと、変更前と変更後が左右に並んで表示されます。ターミナルの `git diff` と同じものなので、見やすい方を使ってかまいません。

5.6　取り消し方

状況	コマンド
保存したけど、まだ add していない変更をなかったことにしたい	`git restore ファイル名`
add したけど、やっぱり準備エリアから外したい	`git restore --staged ファイル名`
直前のコミットのメッセージを直したい（**push する前だけ**）	`git commit --amend -m "新しいメッセージ"`

push した後の履歴は書き換えない

`--amend` や `git reset` で履歴を書き換えたあとに push しようとすると、拒否されます。そこで `git push -f`（強制 push）を使うと、GitHub 側の履歴を上書きしてしまい、チーム開発では他の人の作業を消す事故になります。**強制 push は、三角に相談してからでないと使わない**と決めておいてください。

5.7　.gitignore：Git に記録させないもの

パスワードや、自動で作られる大量のファイルは、Git に記録してはいけません。記録させないファイルを書いておくのが `.gitignore` です。

`.gitignore`（第 2 巻以降で使う例）

```
# 自動で作られる大量のファイル
node_modules/
.next/

# 秘密の設定（API キーなど）
.env*
!.env.example

# OS が勝手に作るファイル
Thumbs.db
.DS_Store
```

> **⚠ 秘密情報を push してしまったら**
>
> API キーやパスワードを含むファイルを GitHub に push してしまったら、**すぐに三角に連絡してください**。コミットを消しても、一度公開されたキーは誰かに見られた前提で扱う必要があり、キーそのものを作り直す必要があります。隠さずにすぐ言ってくれれば、被害はほぼ防げます。

5.8　よくあるエラー

エラーの一部	原因と対処
`Permission denied (publickey)`	SSH 鍵が GitHub に登録されていないか、HTTPS のアドレスと混ざっている。`ssh -T git@github.com` で確かめる
`Please tell me who you are`	初期設定（`user.name`、`user.email`）をしていない。第 3 章に戻る
`nothing to commit, working tree clean`	記録する変更がない。保存（Ctrl+S）し忘れていないか確認する
`! [rejected] ... (fetch first)`	GitHub 側に、自分の PC にない記録がある。`git pull` してから push する
`not a git repository`	Git の管理下にないフォルダにいる。`pwd` で場所を確認する
（`-m` を付け忘れて VS Code のタブが開いた）	メッセージ入力用のタブ。1 行目にメッセージを書いて保存し、タブを閉じれば commit される

腕だめし：わざと失敗して、元に戻す

スプシ 5-6 `git-practice` で次のことを順にやり、各段階で `git status` の表示がどう変わるかをメモしてください。

1. README を適当に書き換えて保存する → `git restore` で元に戻す
2. `keiko.md` という新しいファイルを作り、add する → `git restore --staged` で準備エリアから外す
3. `keiko.md` を commit する。そのときメッセージをわざと間違える → `--amend` で直してから push する

「どの場所にあるファイルを、どの場所に戻したのか」を、本章の 3 つの場所の図を使って定例で説明します。

できるようになったかチェック

□作業場・準備エリア・ローカルの記録・GitHub の 4 つの関係を図で説明できる

□`status` → `add` → `commit` → `push` を、資料を見ずにできる

□`git diff` の `+` と `-` を読める

□Conventional Commits の形でメッセージを書ける

□強制 push をしてはいけない理由を言える

□秘密情報を push してしまったときの対応を言える

第 6 章　Claude Code と組む

スプシ 6-1〜6-5

6.1　Claude Code とは

Claude Code は、ターミナルの中で動く AI です。チャットの Claude と違うのは、次の 2 点です。

- **あなたの PC のファイルを読んで、書き換えられる**
- **コマンドを実行できる**（`git status` や `npm run dev` など）

つまり、「このフォルダのコードを読んで、ここを直して」と頼むと、本当に直してくれます。強力な分、**何をさせたのかを自分で把握しておく**ことが大事になります。

6.2　起動と終了

STEP 1　練習用リポジトリで起動する

Git Bash

```
$ cd ~/dev/git-practice
$ claude
```

Claude Code は、起動した場所のフォルダを「作業場所」として扱います。必ず作業したいリポジトリに `cd` してから起動してください。ホームフォルダ（`~`）で起動すると、PC 中のファイルが対象になってしまいます。

最初に「このフォルダを信頼しますか」と聞かれたら、自分のリポジトリであることを確かめてから `Yes` を選びます。

入力	効果
普通の文章	Claude への依頼
`/exit`	終了する
`/clear`	会話をリセットする（**別の作業に移るときは必ず**）
`/compact`	これまでの会話を要約して軽くする
`/status`	ログイン状態や残りの使用量を見る
`/help`	コマンドの一覧
`Esc`	Claude の作業を途中で止める
`Esc` を 2 回	前の発言に戻ってやり直す
`Shift`+`Tab`	動作モードを切り替える（下で説明）

6.3 許可を求められたら

Claude Code はファイルを書き換える前や、コマンドを実行する前に、許可を求めてきます。



⚠ 許可する前に読む

- **何をしようとしているか分からないコマンドは、許可しない。**3（No）を選び、「このコマンドは何をするものか説明して」と聞く
- `rm -rf`、`git push -f`、`git reset --hard` が出てきたら、必ず止まって確認する
- 「2. 今後は聞かない」は、`git status` のような**読むだけのコマンド**にだけ使う

`Shift`+`Tab` で切り替えられる主なモードは、次の通りです。

モード	動き	この教材での使い方
通常	書き換えのたびに許可を求める	**基本はこれ**
plan mode	書き換えずに、計画だけ立てる	大きめの作業の前に、方針を相談するとき
accept edits	ファイルの書き換えは許可なしで進める	第 2 巻以降、慣れてから

6.4　CLAUDE.md：Claude への申し送り

リポジトリの一番上に `CLAUDE.md` というファイルを置くと、Claude Code は起動するたびにそれを読みます。「このプロジェクトはこういうルールで作っている」という申し送り帳です。

CLAUDE.md（例：練習用リポジトリに置いてあるもの）

```
# このリポジトリについて
理工体剣道部サイトの練習版。作っているのは未経験から学習中の柴山。

## Claude へのお願い
- 答えのコードをいきなり全部書かず、まず方針と理由を説明する
- 新しい用語が出てきたら、1 行で意味を添える
- 変更したら、どのファイルの何行目を、なぜ変えたかを最後にまとめる
- git push、ファイルの削除、強制系のコマンドは実行前に必ず確認する
```

練習用リポジトリには、三角がこれを用意してあります。本番の `rikotaikendo-web` にも `CLAUDE.md` があり、技術構成やルールが書いてあります。第 5 巻で読みます。

STEP 2　練習用リポジトリを clone して、CLAUDE.md を読む

スプシ 6-3 GitHub にあるリポジトリを丸ごと手元にコピーすることを **clone**（クローン）といいます。

Git Bash

```
$ cd ~/dev
$ git clone git@github.com:yutomi141/rikotaikendo-practice.git
$ cd rikotaikendo-practice
$ ls
CLAUDE.md  README.md  docs
$ cat CLAUDE.md
```

`CLAUDE.md` を読んで、Claude がこのリポジトリでどう振る舞うよう頼まれているかを確かめてください。次に、ここで Claude Code を起動して、教材について質問してみます。

Claude Code にこう頼む

docs/textbook/vol1/chapters/ch05-git.tex を読んで、git add と git commit が分かれている理由を、剣道の稽古にたとえて説明して。

このリポジトリの `main` には、直接 push できない設定になっています。変更を加えるのは第 2 巻からで、そのときはブランチを切って PR を出します。教材が更新されたら、このフォルダで `git pull` すると最新になります。

6.5　頼み方の 5 原則

1. **目的を書く**：何のためにやるのか（「新入生がスマホで見るので」）
2. **範囲を書く**：触ってよいファイル、触ってはいけないファイル
3. **完成の条件を書く**：どうなったら終わりか（「幅 375px で横にはみ出ない」）
4. **小さく頼む**：1 回の依頼で 1 つのこと
5. **理由を聞く**：「なぜそうしたのか」まで答えてもらう

⊗ よくない頼み方

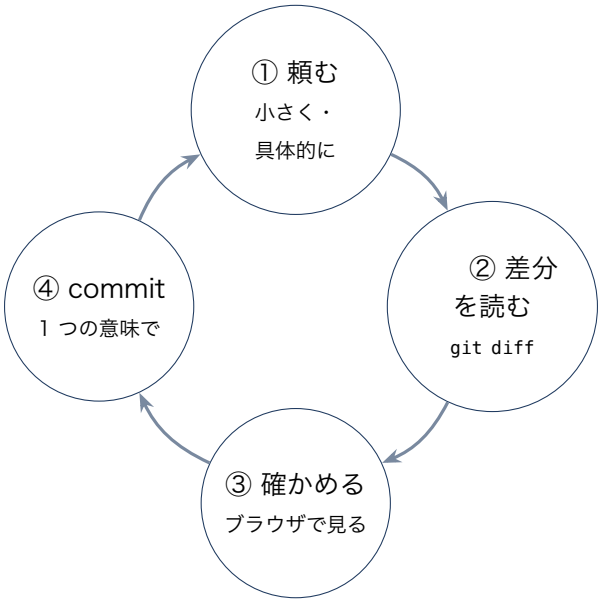
いい感じのトップページを作って

Claude Code にこう頼む

index.html のヒーロー部分（`<section class="hero">`）だけを直したい。
目的：新入生がスマホで開いたとき、部の名前と「稽古見学歓迎」がすぐ目に入るようにしたい。
範囲：index.html と style.css のヒーロー部分だけ。ほかのセクションは触らないで。
完成の条件：幅 375px の画面で、文字が画像からはみ出さないこと。
コードを書く前に、どう直すつもりかと、その理由を先に説明して。

6.6　育てるサイクル

AI との開発は、次のサイクルの繰り返しです。



②と③を飛ばさないことが一番大事です。Claude は「できました」と言いますが、本当にできているかは、あなたが差分を読み、画面で確かめるまで分かりません。③でおかしかったら、commit せずに①に戻ります。うまくいったら④でセーブし、次の小さな依頼に進みます。

6.7 「教えてもらう」使い方

Claude Code は、作業させるだけでなく、先生としても使えます。

分からないことを聞く

いま git diff で出た「@@ -7,4 +7,4 @@」は何を意味している？ 1 つずつ説明して。

自分の理解を確かめる

CSS の position: absolute について、私はこう理解している：「親の要素を基準に、好きな位置に置ける」。
この理解で合っている？ 足りない点や間違いがあれば、例を使って教えて。

エラーを読む練習

このエラーの原因を教えて。ただし、直し方はまだ言わないで。私が先に直し方を考えるから、そのあと答え合わせして。
（エラーメッセージを全文貼る）

6.8　使用量を節約する

Pro の使用量は、Claude が読んだり書いたりする量に比例して減ります。

- **作業が変わったら `/clear`**：前の作業の会話を引きずると、毎回それを読み直すので減りが早くなる
- **範囲を絞る**：「このファイルの、この部分」と指定すると、余計なファイルを読まずに済む
- **大きい作業は plan mode で方針を決めてから**：やり直しが減る
- **簡単なことは自分でやる**：1 文字の修正や `git status` を Claude に頼まない
- `/status` で残量を確認する習慣をつける

上限に達した日にできること

上限に達しても、学習は止まりません。次のことを進めてください。

- 教材を先に読み進める
- 自分で書いたコード、Claude が書いたコードを読み直し、分からない行に印をつけておく（次に聞く）
- 小さな修正を自分の手でやってみる
- 腕だめしの「どこを、どう変えればよいか」のメモを先に書いておく

6.9　AI と付き合う 3 つの掟

3 つの掟

1. **説明できないコードは、commit しない。**分からない行があれば、分かるまで聞く。
2. **秘密は渡さない。**秘密鍵、パスワード、本番の API キーは、Claude にも見せない。
3. **AI の「できました」を信じない。自分の目で確かめる。**

STEP 3　Claude Code で README を育てる

スプシ 6-5 `git-practice` で Claude Code を起動し、次のことを頼んでみましょう。

1. 「README.md に『剣道部での役割』という見出しを足して、箇条書きで 3 つ書けるように枠だけ作って。中身は私が書く」
2. 許可を求められたら、内容を読んでから許可する
3. `/exit` で終了し、`git diff` で何が変わったかを自分で確認する
4. 中身を自分で書き、commit して push する

☑　できるようになったかチェック

□作業したいリポジトリに `cd` してから Claude Code を起動している

□許可を求められたとき、コマンドの意味を確かめてから選べる

□頼み方の 5 原則に沿って依頼を書ける

□育てるサイクル（頼む→差分→確かめる→ commit）を回せる

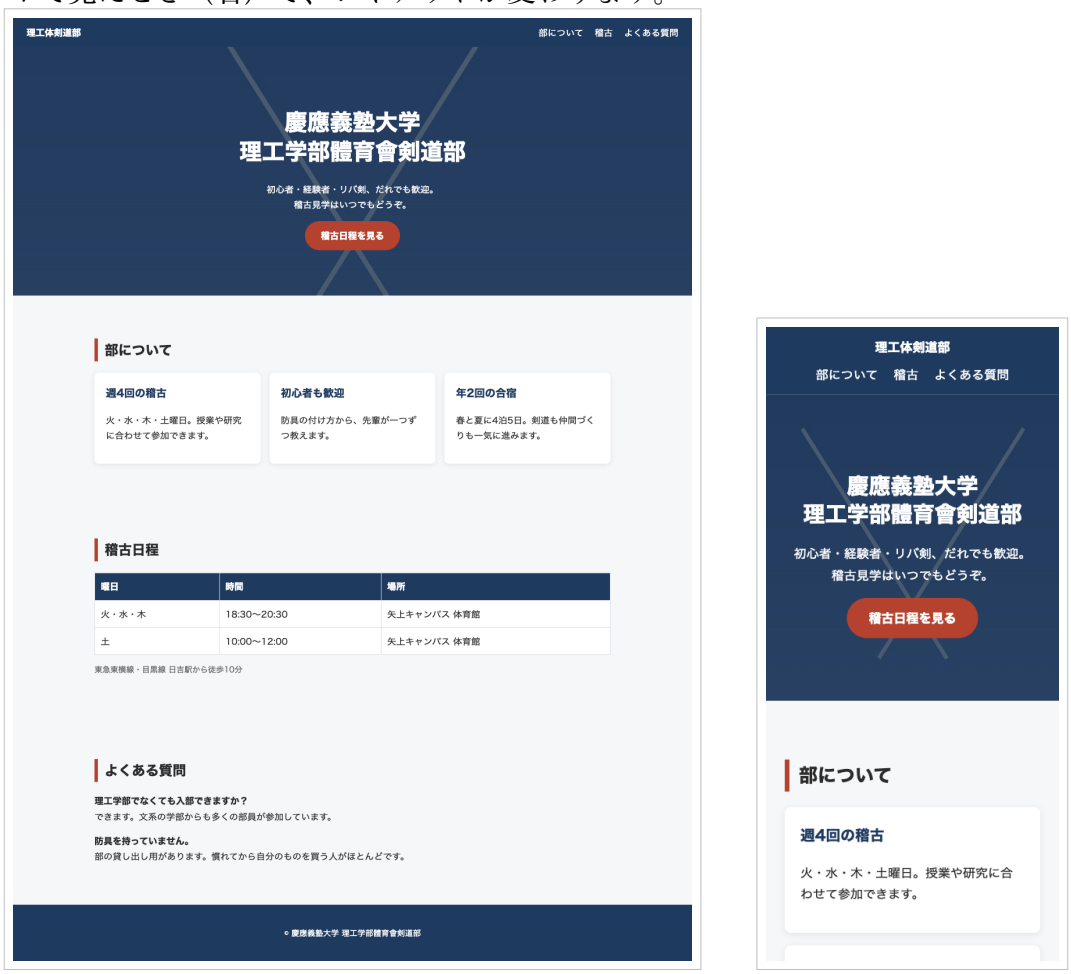
□`/clear` を使う場面を言える

□3 つの掟を言える

第 7 章　フェーズ① HTML と CSS で部のトップページをつくる

スプシ 7-1〜7-10

いよいよ Web ページを作ります。完成形は次のページです。PC で見たとき（左）と、スマホで見たとき（右）で、レイアウトが変わります。



完成イメージ（左：PC　右：スマホ）

この章の進め方

この章のコードは、**Claude に書かせず、自分の手で打ってください。**HTML と CSS は、これから先ずっと読み書きする言葉です。一度自分の手で打っておくと、第 2 巻以降で Claude が書いたコードを読むときの速さが全く違います。打ち間違いでうまく表示されないときに原因を探すのも、大事な練習です。

7.1　準備：リポジトリを作る

STEP 1　kendo-top リポジトリを作る

スプシ 7-1

1. GitHub で `kendo-top` という名前の Public リポジトリを作ります（README は Off）。
2. 手元にフォルダを作り、Git の管理を始めます。

Git Bash

```
$ cd ~/dev
$ mkdir kendo-top
$ cd kendo-top
$ git init
$ mkdir images
$ touch index.html style.css
$ git remote add origin git@github.com:shibayama-ryusei/kendo-top.git
$ code .
```

STEP 2　トップ画像を用意する

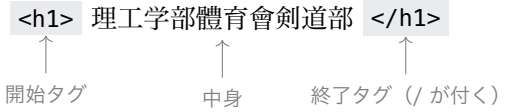
部の集合写真を三角からもらい、`images/hero.jpg` という名前で保存します。

ネットに載せてよい写真か

このリポジトリは Public（誰でも見られる）で、GitHub Pages で公開もします。**部員の顔が写った写真を使うときは、公開してよいものか三角に確認してください。**本番のサイトでも、写真や名前の扱いには同じ注意が必要です。

7.2　HTML の骨組み

HTML は、**タグ**と呼ばれる <名前> の印で文章を挟み、「ここは見出し」「ここは段落」と意味を付けていく言葉です。



STEP 3　骨組みを書く

スプシ 7-2 index.html に次のように書きます。

index.html

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>慶應義塾大学 理工学部體育會剣道部</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- ここにページの中身を書いていく -->
</body>
</html>
```

<head>

ページの「設定」を書く場所。画面には表示されない

<meta charset="UTF-8">

文字化けを防ぐおまじない。日本語を使うなら必須

<meta name="viewport" ...>

スマホで正しい幅で表示させる設定。**これがないとスマホで文字が極小になる**

<title>

ブラウザのタブに出る名前。Google の検索結果にも出る

<link rel="stylesheet" ...>

見た目を決める CSS ファイルを読み込む

<body>

画面に表示される中身を書く場所

<!-- -->

コメント。ブラウザは無視する。メモに使う

7.3　ページを表示して確かめる

STEP 4　ローカルでページを開く

スプシ 7-3 VS Code のターミナル（Ctrl+@）で、次のように打ちます。

Git Bash

```
$ npx serve .
Need to install the following packages:
serve@14.x.x
Ok to proceed? (y) y


   ┌──────────────────────────────────┐
   │   Serving!                       │
   │   - Local:    http://localhost:3000    │
   └──────────────────────────────────┘
```

ブラウザで http://localhost:3000 を開きます。真っ白なページで、タブに部の名前が出ていれば成功です。止めるときは、ターミナルで Ctrl+C を押します。

localhost とは

localhost は「自分の PC」を指す特別な名前です。npx serve は、自分の PC を小さなサーバーにして、第 1 章の図の「サーバー」の役をさせています。:3000 は**ポート番号**で、PC の中の「何番窓口か」を表します。第 2 巻の Next.js でも、同じように localhost:3000 で確かめながら作ります。

7.4　中身を書く

STEP 5　ヘッダーとヒーロー

スプシ 7-4 <body> の中のコメントを消して、次のように書きます。

index.html（<body> の中）

```
  <header class="site-header">
    <p class="logo">理工体剣道部</p>
    <nav>
      <ul class="nav-list">
        <li><a href="#about">部について</a></li>
        <li><a href="#practice">稽古</a></li>
        <li><a href="#faq">よくある質問</a></li>
```

```
      </ul>
    </nav>
  </header>

  <main>
    <section class="hero">
      <h1>慶應義塾大学<br>理工学部體育會剣道部</h1>
      <p>初心者・経験者・リバ剣、だれでも歓迎。<br>稽古見学はいつでもどうぞ。</p>
      <a class="button" href="#practice">稽古日程を見る</a>
    </section>

    <!-- 次の STEP で、ここにセクションを足していく -->
  </main>

  <footer class="site-footer">
    <p>&copy; 慶應義塾大学 理工学部體育會剣道部</p>
  </footer>
```

タグ	意味
`header / main / footer`	ページの頭・本体・足。意味のまとまりを示す
`nav`	サイト内の移動用メニュー
`section`	1 つの話題のまとまり
`h1〜h6`	見出し。数字が小さいほど大きな見出し。**h1 はページに 1 つ**
`p`	段落
`ul / li`	箇条書きの全体／1 項目
`a`	リンク。`href` に行き先を書く。`#about` は同じページの `id="about"` へ飛ぶ
`br`	改行（終了タグなし）

`class="..."` は、CSS で見た目を付けるときの「名札」です。次の節で使います。

STEP 6　部の紹介・稽古日程・よくある質問

スプシ 7-5 `<main>` の中の「次の STEP で…」のコメントを、次の 3 つのセクションに置き換えます。

index.html（`<main>` の中、hero の下）

```
    <section id="about" class="section">
      <h2>部について</h2>
      <div class="cards">
        <article class="card">
          <h3>週 4 回の稽古</h3>
          <p>火・水・木・土曜日。授業や研究に合わせて参加できます。</p>
```

```
        </article>
        <article class="card">
          <h3>初心者も歓迎</h3>
          <p>防具の付け方から、先輩が一つずつ教えます。</p>
        </article>
        <article class="card">
          <h3>年 2 回の合宿</h3>
          <p>春と夏に 4 泊 5 日。剣道も仲間づくりも一気に進みます。</p>
        </article>
      </div>
    </section>

    <section id="practice" class="section">
      <h2>稽古日程</h2>
      <table class="schedule">
        <tr><th>曜日</th><th>時間</th><th>場所</th></tr>
        <tr><td>火・水・木</td><td>18:30〜20:30</td><td>矢上キャンパス 体育館
</td></tr>
        <tr><td>土</td><td>10:00〜12:00</td><td>矢上キャンパス 体育館</td></tr>
      </table>
      <p class="note">東急東横線・目黒線 日吉駅から徒歩 10 分</p>
    </section>

    <section id="faq" class="section">
      <h2>よくある質問</h2>
      <dl class="faq">
        <dt>理工学部でなくても入部できますか？</dt>
        <dd>できます。文系の学部からも多くの部員が参加しています。</dd>
        <dt>防具を持っていません。</dt>
        <dd>部の貸し出し用があります。慣れてから自分のものを買う人がほとんどです。
</dd>
      </dl>
    </section>
```

稽古の時間は例です。実際の時間に書き換えてください。

表とリストのタグ

`table`（表全体）、`tr`（1 行）、`th`（見出しのマス）、`td`（普通のマス）。`dl`・`dt`・`dd` は「用語と説明」の組で、質問と答えの組み合わせにぴったりです。

ブラウザを再読み込み（F5）すると、文字だけの地味なページが表示されます。ここまでで、一度コミットしておきましょう。

```
Git Bash
$ git add index.html images/hero.jpg
$ git commit -m "feat: トップページの HTML を作成"
```

7.5　CSS で見た目を整える

CSS は「どの要素に」「何を」「どうするか」を書く言葉です。



セレクタの書き方は、`.` 名前 で class、`#`名前 で id、`h2` のようにタグ名だけで「そのタグすべて」を指します。`.section h2` のように空白で区切ると、「`.section` の中にある `h2`」という意味になります。

STEP 7　色の約束と全体の設定

スプシ 7-6

style.css

```
/* 色の約束：ここを変えればサイト全体の色が変わる */
:root {
  --color-kon: #1f3a5f;
  --color-shu: #b5422e;
  --color-bg: #f6f7f9;
  --color-text: #2b2b2b;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Hiragino Sans", "Yu Gothic", sans-serif;
  color: var(--color-text);
  background-color: var(--color-bg);
  line-height: 1.7;
}
```

`--color-kon` は **CSS 変数**です。色を 1 か所にまとめておき、`var(--color-kon)` で呼び出します。本番のサイトも、色はこうして 1 か所で管理しています。`#1f3a5f` は

色を 16 進数で表したもので、赤・緑・青の強さを 2 桁ずつ書いています。

STEP 8　ヘッダーとヒーロー

style.css（続き）

```
/* ヘッダー */
.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 24px;
  background-color: var(--color-kon);
  color: white;
}

.logo {
  margin: 0;
  font-weight: bold;
}

.nav-list {
  display: flex;
  gap: 20px;
  margin: 0;
  padding: 0;
  list-style: none;
}

.nav-list a {
  color: white;
  text-decoration: none;
}

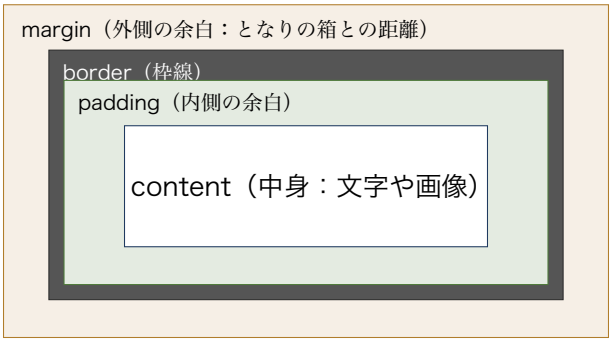
/* ヒーロー（一番上の大きな部分） */
.hero {
  padding: 80px 24px;
  text-align: center;
  color: white;
  background: linear-gradient(rgba(31, 58, 95, 0.75), rgba(31, 58, 95, 0.75)),
    url("images/hero.jpg") center / cover;
}
```

```css
.hero h1 {
  font-size: 40px;
  line-height: 1.3;
}

.button {
  display: inline-block;
  padding: 12px 28px;
  border-radius: 999px;
  background-color: var(--color-shu);
  color: white;
  text-decoration: none;
  font-weight: bold;
}
```

7.6　箱の考え方：ボックスモデル

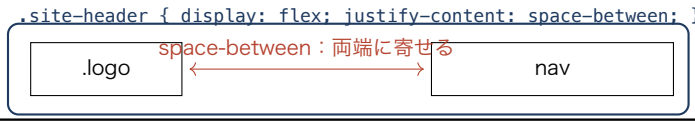
CSS では、すべての要素を「箱」として扱います。箱は内側から順に、次の 4 層でできています。



`padding: 12px 24px;` は「上下 12px、左右 24px の内側の余白」という意味です。最初に書いた `box-sizing: border-box;` は、「幅を指定したら、padding と枠線もその幅の中に含める」という設定です。これがないと、余白を足すたびに箱がはみ出していきます。

7.7　横に並べる：Flexbox

`display: flex;` を指定すると、その箱の**子どもたちが横一列に並びます**。ヘッダーでロゴとメニューが左右に分かれているのは、これを使っているからです。



プロパティ	意味
`justify-content`	並ぶ方向（横）の寄せ方。`space-between` で両端、`center` で中央
`align-items`	並ぶ方向と直角（縦）の揃え方。`center` で上下中央
`gap`	子ども同士の間隔
`flex-direction`	並ぶ向き。`column` にすると縦に並ぶ
`flex: 1`	（子ども側に書く）余った幅を均等に分け合う

STEP 9　セクション・カード・表・フッター

スプシ 7-7

style.css（続き）

```
/* 各セクション共通 */
.section {
  max-width: 960px;
  margin: 0 auto;
  padding: 56px 24px;
}

.section h2 {
  border-left: 6px solid var(--color-shu);
  padding-left: 12px;
}

/* カードを横に並べる */
.cards {
  display: flex;
  gap: 16px;
}

.card {
  flex: 1;
  padding: 20px;
  background-color: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.card h3 {
  margin-top: 0;
```

```
  color: var(--color-kon);
}

/* 稽古日程の表 */
.schedule {
  width: 100%;
  border-collapse: collapse;
  background-color: white;
}

.schedule th,
.schedule td {
  padding: 10px;
  border: 1px solid #d9dde3;
  text-align: left;
}

.schedule th {
  background-color: var(--color-kon);
  color: white;
}

.note {
  font-size: 14px;
  color: #666;
}

/* よくある質問 */
.faq dt {
  font-weight: bold;
  margin-top: 16px;
}

.faq dd {
  margin-left: 0;
}

.site-footer {
  padding: 24px;
  text-align: center;
```

```
  background-color: var(--color-kon);
  color: white;
  font-size: 14px;
}
```

`max-width: 960px; margin: 0 auto;` は、「広い画面でも幅は 960px まで。左右の余白は自動で同じにする（＝中央寄せ）」という定番の書き方です。

7.8　スマホに対応する：レスポンシブ

このままスマホで見ると、カード 3 枚が横に並んだまま押しつぶされます。画面の幅によって CSS を切り替えるのが**メディアクエリ**です。

STEP 10　スマホ用の CSS を足す

スプシ 7-8

style.css（一番最後に足す）

```
/* 画面の幅が 640px 以下（スマホ）のときだけ効く */
@media (max-width: 640px) {
  .site-header {
    flex-direction: column;
    gap: 8px;
  }

  .hero h1 {
    font-size: 28px;
  }

  .cards {
    flex-direction: column;
  }
}
```

新入生はスマホで見る

部のサイトに来る新入生のほとんどは、スマホで見ています。作ったら**必ずスマホの幅で確かめる**ことを習慣にしてください。確かめ方は次の節で説明します。

7.9　開発者ツール

ブラウザ（Chrome または Edge）で `F12` を押すと、**開発者ツール**（DevTools）が開きます。Web 開発で一番お世話になる道具です。



開発者ツールの画面（模式図）。左上のスマホのアイコンで、スマホの幅で表示できる

STEP 11　開発者ツールで 3 つのことを試す

スプシ 7-9

1. **スマホ表示**：`Ctrl`+`Shift`+`M`（またはスマホのアイコン）を押し、上の機種名から `iPhone SE` などを選ぶ。カードが縦に並べば、メディアクエリが効いている
2. **要素を調べる**：左上の矢印アイコンを押してから、ページ上のボタンをクリックする。右側に、そのボタンに効いている CSS が表示される
3. **その場で試す**：右側の CSS の値（例：`padding` の数字）をクリックして書き換えると、すぐに見た目に反映される。ただし**再読み込みすると元に戻る**ので、気に入ったら `style.css` にも書く

```
Git Bash
$ git add style.css
$ git commit -m "style: トップページのデザインとスマホ対応を追加"
$ git push -u origin main
```

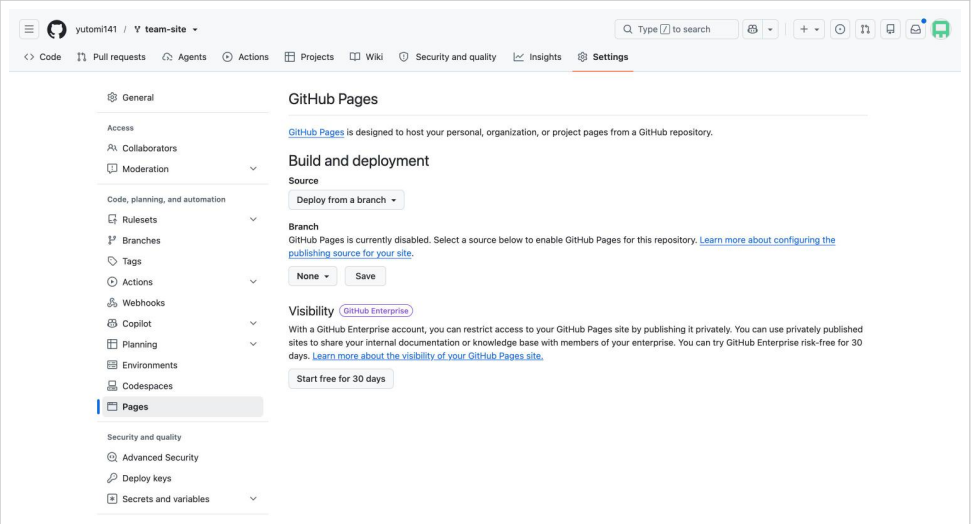
7.10　GitHub Pages で公開する

GitHub Pages を使うと、リポジトリの HTML をそのまま無料でインターネットに公開できます。

STEP 12　Pages を有効にする

スプシ 7-10

1. GitHub の `kendo-top` リポジトリを開き、**Settings** → 左側の **Pages** を押します。
2. **Branch** を `None` から `main` に変え、右のフォルダは `/ (root)` のまま Save を押します。



3. 1〜2 分待って再読み込みすると、上に `Your site is live at https://(ユーザー名).github.io/kendo-top/` と表示されます。
4. そのアドレスをスマホで開いて、表示を確かめます。

公開されないときのチェック

- ファイル名が `index.html`（すべて小文字）になっているか
- `index.html` がリポジトリの一番上の階層にあるか（フォルダの中に入っていないか）
- 画像が表示されないなら、ファイル名の大文字・小文字が HTML の記述と完全

に一致しているか（`hero.JPG` と `hero.jpg` は別物扱い）

- リポジトリの **Actions** タブで、`pages build and deployment` が緑色になっているか

7.11　練習問題

スプシ 7-12〜7-27

ここまでの内容を、手を動かして固めます。**★が多いほど難しい**問題です。練習用に、`kendo-top` とは別に `~/dev/html-drill` というフォルダを作り、問題ごとに `q01.html`、`q02.html` …とファイルを分けて解いてください（`git init` して、解けたらコミットする）。解答例は付録にあります。**答えを見るのは、自分で書いてブラウザで確かめてから**にしてください。

練習問題での Claude の使い方

練習問題では、Claude に答えを書かせません。使ってよいのは次の 2 つだけです。

- 「`<dl>` タグの使い方を教えて。ただしこの問題の答えは書かないで」のように、**道具の使い方**を聞く
- 自分の答えを見せて「どこが間違っているか、ヒントだけちょうだい」と**答え合わせ**をする

HTML

練習 7.1　見出しの階層

→ 解答例は付録

次の文章を、`h1`〜`h3` と `p` を使って HTML にしてください。

理工体剣道部の一年（大見出し）
春（中見出し）　新歓（小見出し）：新入生の体験稽古を行う。　早慶戦（小見出し）：伝統の一戦。
夏（中見出し）　夏合宿（小見出し）：4 泊 5 日で鍛える。

練習 7.2　外部リンク

→ 解答例は付録

「部の Instagram はこちら」という文章を書き、「こちら」を押すと `https://www.instagram.com/` が**新しいタブで**開くようにしてください。新しいタブで開くときに一緒に付けるべき属性も調べて付けてください。

練習 7.3　画像と代わりの文章 ★

→ 解答例は付録

`images/hero.jpg` を幅 300px で表示してください。あわせて、画像が読み込めなかったときや、目の見えない人が読み上げソフトを使ったときに伝わる説明文を付けてください。

練習 7.4　持ち物リスト ★★

→ 解答例は付録

「体験稽古の持ち物」として、動きやすい服装・タオル・飲み物・上履き（体育館用）の 4 つを箇条書きにしてください。次に、「入部までの流れ」（見学する → 体験稽古に出る → 入部届を出す）を、**番号付き**の箇条書きにしてください。2 つでタグをどう使い分けたかを説明できるようにしておきます。

練習 7.5　年間行事の表 ★★

→ 解答例は付録

次の表を HTML で作ってください。見出しの行は `<thead>`、中身は `<tbody>` に入れます。

月	行事	区分
4 月	新歓・体験稽古	イベント
6 月	早慶戦	試合
8 月	夏合宿	合宿
3 月	春合宿	合宿

練習 7.6　間違い探し ★★

→ 解答例は付録

次の HTML には、間違いが**4 つ**あります。すべて見つけて直してください。

q06.html

```
<section>
  <h2>稽古について<h2>
  <p>稽古は<strong>週 4 回</p></strong>
  <ul>
    <p>火曜日</p>
    <li>水曜日</li>
  </ul>
  <img src="images/hero.jpg">
</section>
```

練習 7.7　問い合わせフォーム（見た目だけ）★★

→ 解答例は付録

名前・メールアドレス・問い合わせ内容（複数行）・送信ボタンの 4 つを持つフォームを作ってください。

- 各入力欄には `<label>` を付け、ラベルを押すとその入力欄にカーソルが入るようにする
- メールアドレス欄は、メールの形になっていないと送信できないようにする
- 名前とメールアドレスは、空のままでは送信できないようにする

（実際の送信の仕組みは第 5 巻で作ります。ここでは見た目と入力のチェックだけです。）

CSS

練習 7.8　サイトの色を一気に変える★

→ 解答例は付録

`kendo-top` の `style.css` で、**`:root` の中だけを書き換えて**、紺の部分を深緑（`#1e4d3a`）に、朱の部分を金色（`#b8860b`）に変えてください。確かめたら、`git restore style.css` で元に戻します。CSS 変数を使うと何がうれしいのかを、1 文で説明できるようにしておきます。

練習 7.9　セレクタはどれに効く？★

→ 解答例は付録

次の HTML と CSS で、**赤くなる文字**はどれでしょう。ブラウザで確かめる前に予想を書いてください。

q09.html

```
<section class="news">
  <h2>お知らせ</h2>
  <p>新歓稽古を行います。</p>
  <p class="important">雨天でも実施します。</p>
</section>
<p class="important">このサイトは練習用です。</p>

<style>
```

```
  .news p { color: red; }
  #top { color: red; }
  h3 { color: red; }
</style>
```

練習 7.10　箱の幅を計算する ★★

→ 解答例は付録

次の CSS が付いた `div` の、**画面上での横幅**（枠線の外側まで）は何 px でしょう。(1) `box-sizing` の指定がない場合と、(2) `box-sizing: border-box;` がある場合の両方を計算し、開発者ツールで実際の数字を確かめてください。

```
div { width: 300px; padding: 20px; border: 2px solid black; margin: 10px; }
```

練習 7.11　ボタンに反応をつける ★★

→ 解答例は付録

`.button` にマウスを乗せたとき、少し暗い色になり、0.2 秒かけてなめらかに色が変わるようにしてください。（ヒント：`:hover` と `transition`）

練習 7.12　フッターを 3 列にする ★★

→ 解答例は付録

フッターを、「部の名前」「稽古場所」「SNS」の 3 つのかたまりが横に並ぶ形に作り変えてください。スマホの幅では縦 1 列に並ぶようにします。

練習 7.13　どちらの CSS が勝つ？ ★★★

→ 解答例は付録

次の 3 つのルールが同じ `<h2 class="title" id="main-title">` に当たったとき、文字は何色になるでしょう。理由を「詳細度」という言葉を使って説明してください。

```
#main-title { color: green; }
h2.title    { color: blue; }
.title      { color: red; }
```

そのうえで、3 つ目の `.title` の色を勝たせるにはどう書き換えればよいかを考えてください（`!important` は使わない）。

レスポンシブと開発者ツール

練習 7.14　タブレットでは 2 列にする ★★

→ 解答例は付録

「部について」のカードを、画面の幅に応じて次のように並べてください。

画面の幅	カードの並び
961px 以上（PC）	横に 3 枚
641〜960px（タブレット）	2 枚ずつ（3 枚目は次の行）
640px 以下（スマホ）	縦に 1 枚ずつ

（ヒント：`flex-wrap: wrap;` と、カードの幅の指定）

練習 7.15　開発者ツールで原因を探す ★★

→ 解答例は付録

`kendo-top` の見出し（`h2`）の左にある朱色の線について、開発者ツールだけを使って次のことを調べてください。

1. その線は、`style.css` の何行目の、どのプロパティで指定されているか
2. 線の太さを開発者ツール上で 12px に変えると、どう見えるか
3. 再読み込みすると元に戻ることを確かめる

練習 7.16　見本を再現する ★★★

→ 解答例は付録

次の見本と同じ見た目の「部の数字」セクションを、HTML と CSS で作ってください。数字は大きく朱色、説明は小さく灰色です。スマホでは 2 枚ずつ並べます。



（数字はダミーです）

腕だめし：「大会結果」のセクションを足す

スプシ 7-28 次の条件で、「稽古日程」と「よくある質問」の間に「最近の大会結果」セクションを追加してください。

1. 大会名・日付・結果の 3 列の表にする（内容はダミーでよい。例：「関東理工科

大学剣道大会　2026 年 10 月　団体戦 ベスト 8」を 3 行）

2. ヘッダーのメニューにも「大会結果」を足し、押すとそのセクションに飛ぶ
3. **スマホの幅で見たときに、表がはみ出して画面全体が横にずれないこと**（ヒント：表を `div` で囲み、その `div` に `overflow-x: auto;` を付ける）
4. 1 つの意味ごとにコミットして push し、GitHub Pages で確かめる

手を動かす前に、「どのファイルの、どこに、何を足すか」をスプシの詰まりメモ欄に箇条書きで書いてください。終わったら、書いたメモと実際にやったことがどう違ったかを、定例で説明します。

☑ できるようになったかチェック

- □`<head>` と `<body>` の役割の違いを言える
- □class と id の使い分けを説明できる
- □CSS のセレクタ・プロパティ・値を指して説明できる
- □ボックスモデルの 4 層（margin・border・padding・content）を図で描ける
- □Flexbox で要素を横に並べられる
- □メディアクエリでスマホ用の見た目を切り替えられる
- □開発者ツールで、要素に効いている CSS を調べられる
- □GitHub Pages で公開し、スマホで確かめた

第 8 章　フェーズ①’ JavaScript で動きをつける

スプシ 8-1〜8-12

第 7 章のページは、見た目は整いましたが、まだ何も「動き」ません。この章では JavaScript を使って、次の 2 つを作ります。



左：学年のボタンで部員を絞り込む　右：質問を押すと答えが開く

どちらも、**データ（配列）を用意して、そこから画面を作る**という形で作ります。これは、第 2 巻の React でも、第 3 巻のデータベースでも、本番のサイトでも、ずっと同じ形です。この章で一番身につけてほしい考え方です。

8.1　JavaScript を動かしてみる

JavaScript は、ブラウザの中で動くプログラミング言語です。まずはファイルを作らずに、ブラウザの**コンソール**で動かしてみます。

STEP 1　コンソールで計算する

スプシ 8-1 ブラウザで `F12` を押し、開発者ツールの **Console** タブを開きます。`>` の横に次のように打ち、`Enter` を押します。

```
Console
> 1 + 2
3
> "面" + "打ち"
'面打ち'
> console.log("稽古開始")
稽古開始
```

貼り付けが拒否されたら

コンソールに初めてコードを貼り付けると、安全のために「allow pasting と入力してください」と表示されることがあります。これは、知らない人に言われるままコードを貼って乗っ取られる詐欺を防ぐための仕組みです。この教材のコードは自分で打つので、表示された通りに入力して進めてかまいません。

8.2 変数とデータの型

変数は、値に名前をつけて取っておく箱です。

```
const clubName = "理工体剣道部";   // 変えない値は const
let count = 0;                     // あとで変える値は let
count = count + 1;
console.log(clubName, count);      // → 理工体剣道部 1
```

const と let の使い分け

基本は `const`、あとで値を入れ直すときだけ `let` を使います。古い資料に出てくる `var` は使いません。

型	例	説明
文字列（string）	`"面"`、`'小手'`	文字。`"` か `'` で囲む
数値（number）	`4`、`18.5`	計算できる数
真偽値（boolean）	`true`、`false`	はい／いいえ
配列（array）	`["面", "小手", "胴"]`	値の並び。順番がある
オブジェクト（object）	`{ name: "上田", grade: 2 }`	名前付きの値のまとまり

”2” と 2 は別物

`"2"` は文字列、`2` は数値です。見た目は同じでも、`"2" === 2` は `false` になります。画面のボタンから取り出した値はたいてい文字列なので、数値と比べるときは `Number("2")` で数値に変換します。この章の絞り込みで、実際にこの変換が出てきます。

8.3 配列とオブジェクト

部員 1 人分の情報は、オブジェクトで表せます。部員全員は、それを並べた配列で表せます。

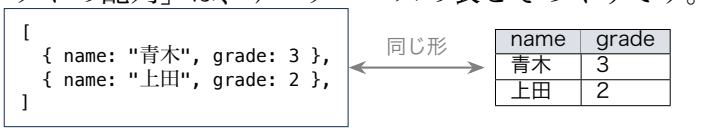
```
const members = [
  { name: "青木 太郎", grade: 3, dan: "三段", role: "主将" },
  { name: "上田 健",   grade: 2, dan: "二段", role: "" },
];

console.log(members.length);     // → 2（人数）
console.log(members[0].name);    // → 青木 太郎（0 番目の人の名前）
console.log(members[1].grade);   // → 2
```

? 配列は 0 番から数える

配列の最初の要素は `[0]`、2 番目は `[1]` です。最初は戸惑いますが、ほぼすべてのプログラミング言語がこの数え方です。

この「オブジェクトの配列」は、データベースの表とそっくりです。



JavaScript の配列　　データベースの表（第 3 巻）

8.4 関数

関数は、処理に名前をつけて、何度でも呼び出せるようにしたものです。

```
// 受け取った部員の紹介文を返す関数
function introduce(member) {
  return `${member.grade}年の${member.name}です`;
}

console.log(introduce(members[0]));   // → 3 年の青木 太郎です

// 同じ関数を「アロー関数」で書くと、こうなる
const introduce2 = (member) => `${member.grade}年の${member.name}です`;
```

`（バッククォート）で囲んだ文字列は**テンプレートリテラル**といい、`${ }` の中に変数を埋め込めます。Shift+@ で入力できます。

8.5 条件分岐と繰り返し

```
// if：条件によって処理を分ける
const member = members[0];
if (member.role === "主将") {
  console.log("主将です");
```

```
} else {
  console.log("部員です");
}

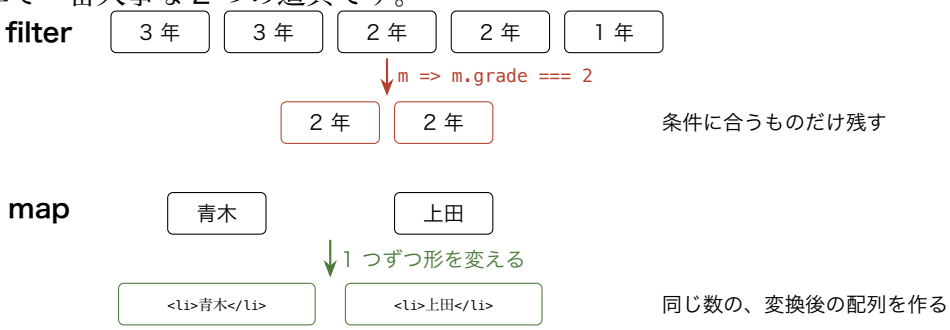
// forEach：配列の 1 人ずつに同じ処理をする
members.forEach((member) => {
  console.log(member.name);
});
```

=== を使う

「等しいか」を調べるときは `===`（イコール 3 つ）を使います。`==`（2 つ）は型を勝手に変換して比べるので、思わぬ結果になることがあります。

8.6　配列を変換する：filter と map

この章で一番大事な 2 つの道具です。




```
// 2 年生だけを取り出す
const second = members.filter((member) => member.grade === 2);

// 名前だけの配列にする
const names = members.map((member) => member.name);   // → ["青木 太郎", "上田
 健"]

// HTML の文字列に変換して、つなげる
const html = members.map((member) => `<li>${member.name}</li>`).join("");
```

8.7　DOM：JavaScript から HTML を操作する

ブラウザは、読み込んだ HTML を **DOM** という「木」の形で覚えています。JavaScript はこの木の枝を探して、中身を書き換えたり、クリックを待ち受けたりできます。

書き方	すること
`document.querySelector(".a")`	CSS と同じ書き方で、最初に見つかった 1 つの要素を取ってくる
`document.querySelectorAll(".a")`	当てはまる要素を全部取ってくる
`要素.textContent = "..."`	中の文字を書き換える
`要素.innerHTML = "..."`	中の HTML を書き換える
`要素.classList.add("x")`	class を足す（`remove` で外す、`toggle` で付け外しを切り替える）
`要素.addEventListener("click", 関数)`	クリックされたら関数を実行する
`要素.dataset.grade`	HTML の `data-grade="2"` の値を取り出す

8.8　作ってみる

STEP 2　HTML に部員紹介の枠を足し、JavaScript を読み込む

スプシ 8-6 `kendo-top` の `index.html` で、「よくある質問」のセクションを、次の 2 つのセクションに**置き換えます**。質問と答えは、HTML に直接書くのをやめ、JavaScript のデータから作ります。

index.html（「よくある質問」のセクションを置き換え）

```
    <section id="members" class="section">
      <h2>部員紹介</h2>
      <div class="filter">
        <button class="filter-button is-active" data-grade="all">全員</button>
        <button class="filter-button" data-grade="1">1 年</button>
        <button class="filter-button" data-grade="2">2 年</button>
        <button class="filter-button" data-grade="3">3 年</button>
      </div>
      <p class="member-count"></p>
      <ul class="member-list"></ul>
    </section>

    <section id="faq" class="section">
      <h2>よくある質問</h2>
      <div class="faq-list"></div>
    </section>
```

さらに、`</body>` の直前に次の 2 行を足します。

index.html（</body> の直前）

```
  <script src="data.js"></script>
  <script src="script.js"></script>
```

⑦ script を body の最後に置く理由

ブラウザは HTML を上から順に読みます。`<script>` を `<head>` に置くと、まだ部員紹介の枠（`.member-list`）が作られる前に JavaScript が動いてしまい、「枠が見つからない」というエラーになります。一番最後に置けば、枠ができあがった後に動きます。

STEP 3　データのファイルを作る

スプシ 8-7

data.js

```
// 部員のデータ（名前はダミー）
const members = [
  { name: "青木 太郎", grade: 3, dan: "三段", role: "主将" },
  { name: "井上 花子", grade: 3, dan: "二段", role: "女子主将" },
  { name: "上田 健", grade: 2, dan: "二段", role: "" },
  { name: "江藤 美咲", grade: 2, dan: "初段", role: "" },
  { name: "小川 翔", grade: 1, dan: "無", role: "" },
  { name: "加藤 葵", grade: 1, dan: "初段", role: "" },
];

// よくある質問のデータ
const faqs = [
  {
    question: "理工学部でなくても入部できますか？",
    answer: "できます。文系の学部からも多くの部員が参加しています。",
  },
  {
    question: "防具を持っていません。",
    answer: "部の貸し出し用があります。慣れてから自分のものを買う人がほとんどです。",
  },
  {
    question: "途中から入部してもいいですか？",
    answer: "もちろんです。秋から入部する人もいます。",
  },
];
```

名前はダミーです。このリポジトリは公開されるので、本物の部員の名前は入れません。

⑦ データと処理を別のファイルに分ける理由

部員が入れ替わったときに直すのは `data.js` だけで済み、処理の `script.js` には触らずに済みます。本番のサイトでは、このデータの置き場所がデータベースになり、管理画面から書き換えられるようになります。「データと処理を分ける」という考え方が、そのまま本番の仕組みにつながっています。

STEP 4　よくある質問を作り、クリックで開閉させる

スプシ 8-8

script.js

```
// ===== よくある質問：データから HTML を作り、クリックで開閉する =====
const faqList = document.querySelector(".faq-list");

faqs.forEach((faq) => {
  const item = document.createElement("div");
  item.className = "faq-item";
  item.innerHTML = `
    <button class="faq-question">${faq.question}</button>
    <p class="faq-answer">${faq.answer}</p>
  `;
  faqList.appendChild(item);
});

const questionButtons = document.querySelectorAll(".faq-question");

questionButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const item = button.parentElement;
    item.classList.toggle("is-open");
  });
});
```

1 行ずつ読むと、次のことをしています。

1. `.faq-list` の枠を取ってくる
2. `faqs` の 1 つずつについて、`div` を作り、中に質問のボタンと答えの段落を入れて、枠に足す
3. できあがった質問のボタンを全部取ってくる
4. それぞれのボタンに「押されたら、親の `div` の `is-open` を付け外しする」という仕掛けをつける

開いた・閉じたの見た目は、次の STEP の CSS で `is-open` が付いているかどうかによって切り替えます。**JavaScript は class を付け外しするだけ、見た目は CSS に任せる**、という役割分担です。

STEP 5　部員紹介を作り、学年で絞り込む

スプシ 8-9 `script.js` の続きに書きます。

script.js（続き）

```
// ===== 部員紹介：学年で絞り込む =====
const memberList = document.querySelector(".member-list");
const memberCount = document.querySelector(".member-count");
const filterButtons = document.querySelectorAll(".filter-button");

function renderMembers(grade) {
  // "all" なら全員、それ以外は学年が一致する人だけ残す
  const shown =
    grade === "all"
      ? members
      : members.filter((member) => member.grade === Number(grade));

  memberList.innerHTML = shown
    .map((member) => {
      const role = member.role ? `<span class="role">${member.role}</span>` \
+     ↪ : "";
      return `<li \
+     ↪ class="member">${role}<strong>${member.name}</strong><br>${member.grade}
年・${member.dan}</li>`;
    })
    .join("");

  memberCount.textContent = `${shown.length}人`;
}

filterButtons.forEach((button) => {
  button.addEventListener("click", () => {
    filterButtons.forEach((b) => b.classList.remove("is-active"));
    button.classList.add("is-active");
    renderMembers(button.dataset.grade);
  });
```

```
});

// 最初は全員を表示する
renderMembers("all");
```

`renderMembers` は「学年を受け取って、その学年の部員で画面を描き直す」関数です。ボタンが押されるたびに、押されたボタンの `data-grade` を渡して呼び出しています。`data-grade` の値は文字列なので、`Number(grade)` で数値に変換してから `member.grade` と比べています。

STEP 6　CSS を足す

スプシ 8-10 `style.css` の `/* よくある質問 */` のルール（`.faq dt` と `.faq dd`）を消して、代わりに次を足します。

style.css

```
/* 部員紹介 */
.filter {
  display: flex;
  gap: 8px;
}

.filter-button {
  padding: 6px 16px;
  border: 1px solid var(--color-kon);
  border-radius: 999px;
  background-color: white;
  color: var(--color-kon);
  cursor: pointer;
}

.filter-button.is-active {
  background-color: var(--color-kon);
  color: white;
}

.member-list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 12px;
  padding: 0;
  list-style: none;
```

```
}

.member {
  padding: 12px;
  background-color: white;
  border-radius: 8px;
}

.role {
  display: inline-block;
  margin-right: 6px;
  padding: 0 8px;
  border-radius: 4px;
  background-color: var(--color-shu);
  color: white;
  font-size: 12px;
}

/* よくある質問（開閉） */
.faq-item {
  border-bottom: 1px solid #d9dde3;
}

.faq-question {
  width: 100%;
  padding: 14px 0;
  border: none;
  background: none;
  text-align: left;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
}

.faq-question::after {
  content: "＋";
  float: right;
}

.faq-answer {
  display: none;
  margin-top: 0;
```

```
}

.faq-item.is-open .faq-answer {
  display: block;
}

.faq-item.is-open .faq-question::after {
  content: "－";
}
```

`display: grid;` は Flexbox の仲間で、**縦横のマス目**に並べるときに使います。`repeat(auto-fill, minmax(180px, 1fr))` は「1 マス最低 180px で、入るだけ並べる」という意味で、メディアクエリを書かなくても画面幅に合わせて列の数が自動で変わります。

ブラウザで動きを確かめたら、コミットして push します。

```
Git Bash
$ git add index.html data.js script.js style.css
$ git commit -m "feat: よくある質問の開閉と、部員の学年絞り込みを追加"
$ git push
```

8.9　エラーの読み方

JavaScript でつまずいたら、まず開発者ツールの **Console** を見ます。赤い文字がエラーです。

```
Console
Uncaught TypeError: Cannot read properties of null (reading 'appendChild')
    at script.js:11:11
```

部分	読み方
`TypeError`	エラーの種類。「型が想定と違う」
`Cannot read properties of null`	「`null`（何もない）の中身を読もうとした」
`(reading 'appendChild')`	読もうとしたのは `appendChild`
`script.js:11:11`	**`script.js` の 11 行目**。クリックするとその場所が開く

つまり、「11 行目で `appendChild` しようとした相手が `null`（見つからなかった）」ということです。よくある原因は、`querySelector` に渡したクラス名の打ち間違い（`.faq-lsit` など）か、`<script>` を置く場所が早すぎることです。

よく出るエラー	よくある原因
`X is not defined`	変数名の打ち間違い。大文字と小文字も区別される
`Cannot read properties of null`	`querySelector` で要素が見つかっていない
`Cannot read properties of undefined`	配列の範囲外（`members[10]`）や、存在しない名前（`member.nmae`）を読んだ
`Unexpected token`	かっこやカンマの閉じ忘れ・付けすぎ
`X is not a function`	関数でないものを `()` で呼んだ。名前の打ち間違いが多い

エラーを Claude に聞くとき

script.js を実行したら、コンソールにこのエラーが出た。
（エラーを全文貼る）
私は、11 行目の faqList が null になっているのが原因だと思っている。この読みが合っているかと、なぜ null になるのかを教えて。直したコードはまだ出さないで。

8.10　練習問題

スプシ 8-13〜8-24

`~/dev/js-drill` フォルダを作り、問題ごとにファイル（`q01.js` など）を分けて解きます。1〜9 は、`node q01.js` のように Git Bash で実行して結果を確かめられます（Node.js は第 3 章で入れました）。10〜12 は HTML と組み合わせるので、ブラウザで確かめます。

node で実行する

Git Bash

```
$ cd ~/dev/js-drill
$ node q01.js
```

`console.log` で出したものが、そのままターミナルに表示されます。

基本

練習 8.1　稽古の合計時間

→ 解答例は付録

1 回の稽古が 120 分、週に 4 回、前期が 15 週あります。前期の稽古時間の合計を「分」で計算して表示してください。数字はすべて `const` の変数に入れ、計算式の中に数字を直接書かないこと。

練習 8.2　型を当てる ★

→ 解答例は付録

次のそれぞれを `console.log` すると何が表示されるか、**先に予想を書いてから**実行してください。

```
console.log(typeof "4");
console.log(typeof 4);
console.log("4" + 4);
console.log(4 + 4);
console.log("4" === 4);
console.log(Number("4") === 4);
```

練習 8.3　技の配列 ★

→ 解答例は付録

`["面", "小手", "胴"]` という配列を作り、次の順に操作して、そのたびに配列全体と長さを表示してください。(1) 最後に `"突き"` を足す　(2) 2 番目の要素（`"小手"`）だけを表示する　(3) 最後の要素を、長さを使って取り出す

練習 8.4　自分のプロフィール ★

→ 解答例は付録

自分の名前・学年・学科・好きな技をオブジェクトにまとめ、テンプレートリテラルを使って「応用化学科 2 年の柴山です。好きな技は出小手です。」のような文を表示してください。

練習 8.5　分を「時間と分」に直す関数 ★★

→ 解答例は付録

分を受け取って、`"2 時間 30 分"` のような文字列を返す関数 `formatMinutes` を作ってください。`formatMinutes(150)` → `"2 時間 30 分"`、`formatMinutes(60)` → `"1 時間 0 分"`、`formatMinutes(45)` → `"0 時間 45 分"`。（ヒント：割り算の余りは `%`、小数の切り捨ては `Math.floor`）余裕があれば、`"1 時間"`、`"45 分"` のように 0 の部分を省略するようにしてください。

練習 8.6　上級生と下級生 ★★

→ 解答例は付録

学年を受け取り、1 年なら `"新入生"`、2 年なら `"中堅"`、3 年以上なら `"上級生"`、それ以外（0 や負の数）なら `"学年が正しくありません"` を返す関数を作り、5 つの値で試してください。

配列の変換

以下の問題では、本章の `data.js` の `members` を、ファイルの先頭にコピーして使います。

練習 8.7　段位を持っている人 ★★

→ 解答例は付録

段位が `"無"` でない部員だけを取り出し、その人数と名前を表示してください。

練習 8.8　さん付けの名簿 ★★

→ 解答例は付録

`map` を使って、`["青木 太郎さん（3 年）", "井上 花子さん（3 年）", ...]` という配列を作ってください。次に、`join` を使って、それを改行区切りの 1 つの文字列にして表示してください。

練習 8.9　学年ごとの人数 ★★★

→ 解答例は付録

学年ごとの人数を数え、`{ 1: 2, 2: 2, 3: 2 }` のようなオブジェクトを作って表示してください。学年が増えても（4 年がいても）コードを書き換えずに済むようにすること。

画面を動かす

練習 8.10　素振りカウンター ★★

→ 解答例は付録

「素振り +1」ボタンと「リセット」ボタン、回数を表示する場所を作ってください。「+1」を押すたびに回数が 1 ずつ増え、100 回になったら文字の色が朱色に変わるようにします。（ヒント：色は JavaScript で直接変えず、CSS の class を付け外しする）

練習 8.11　名前で検索する ★★★

→ 解答例は付録

本章の部員紹介に、名前の検索欄（`<input type="text">`）を足してください。文字を打つたびに、その文字を名前に含む部員だけが表示されるようにします。（ヒント：`input` イベントと、文字列の `includes`）

練習 8.12　バグを直す ★★★

→ 解答例は付録

次のコードは、2 年生の名前だけを画面に表示するつもりで書かれていますが、**バグが 3 つ**あります。コンソールのエラーを読みながら、すべて直してください。直した理由もそれぞれ 1 行で書きます。

q12.js

```
const list = document.querySelector(".member-lsit");
const grade = "2";

const second = members.filter((member) => member.grade === grade);

list.innerHTML = second.map((member) => `<li>${member.nmae}</li>`).join("");
```

腕だめし：部員紹介に「役職」と「段位」の絞り込みを足す

スプシ 8-25

1. 学年のボタンの下に、「役職のある人だけ」というチェックボックスを付ける。チェックを入れると、役職（`role`）が空でない人だけが表示される
2. **学年とチェックボックスは同時に効く**こと（例：3 年 ＋ 役職のある人だけ → 主将と女子主将の 2 人）
3. 並び順を「学年の高い順、同じ学年なら名前順」にする（ヒント：`sort`）
4. `data.js` にダミーの部員を 4 人足し、表示や人数が正しく変わることを確かめる

手を動かす前に、「`renderMembers` のどこを、どう変えれば、2 つの条件を同時に効かせられるか」をメモに書いてください。定例では、そのメモと実際のコードを並べて説明します。

できるようになったかチェック

- □`const` と `let` を使い分けられる
- □文字列の `"2"` と数値の `2` が違うことを説明できる
- □オブジェクトの配列から、`filter` と `map` で必要な形を作れる
- □`querySelector` と `addEventListener` で、クリックに反応させられる
- □「JavaScript は class を付け外しするだけ、見た目は CSS」の役割分担を説明できる

□「データから画面を作る」とはどういうことかを、部員紹介を例に説明できる

□コンソールのエラーから、ファイル名と行番号を読み取れる

8.11　第 1 巻のおわりに

おつかれさまでした。ここまでで、次のことができるようになりました。

・Windows に開発環境を整え、Git と GitHub でコードを管理する

・Claude Code に、範囲と完成の条件をはっきりさせて頼み、差分を読んで確かめる

・HTML・CSS・JavaScript で、データから画面を作り、インターネットに公開する

第 2 巻では、この章で作った部員紹介と FAQ を、本番のサイトと同じ **Next.js** で作り直します。`members.filter(...).map(...)` の形が、ほとんどそのまま出てきます。

付録 A　コマンド早見表

ターミナル（Git Bash）

コマンド	すること
`pwd`	いま居る場所を表示
`ls` / `ls -a`	中身の一覧／隠しファイルも含めた一覧
`cd 場所` / `cd ..` / `cd`	移動／1 つ上へ／ホームへ
`mkdir 名前`	フォルダを作る
`touch 名前`	空のファイルを作る
`cat ファイル`	中身を表示
`cp 元 先` / `mv 元 先`	コピー／移動・名前変更
`rm ファイル`	削除（ゴミ箱に入らない）
`code .` / `explorer .`	VS Code／エクスプローラーで開く
`clear`	画面をきれいにする

Git

コマンド	すること
`git init`	ここで Git の管理を始める
`git status`	いまの状態を見る（迷ったらこれ）
`git add ファイル`	変更を準備エリアに入れる
`git commit -m "メッセージ"`	記録する
`git push`	GitHub に送る（初回は `git push -u origin main`）
`git pull`	GitHub の変更を取り込む
`git clone アドレス`	GitHub のリポジトリを丸ごと手元にコピーする
`git diff`	まだ add していない変更を見る
`git diff --staged`	add 済みの変更を見る
`git log --oneline`	履歴を 1 行ずつ見る（`q` で抜ける）
`git restore ファイル`	add 前の変更を取り消す
`git restore --staged ファイル`	準備エリアから外す
`git commit --amend -m "..."`	直前のコミットを直す（push 前だけ）
`git remote -v`	つながっている GitHub のアドレスを見る

Claude Code

入力	すること
`claude`	起動（作業したいリポジトリで）
`/clear`	会話をリセット
`/compact`	会話を要約して軽くする
`/status`	ログイン状態・使用量
`/exit`	終了
`Esc` / `Esc`×2	作業を止める／前の発言に戻る
`Shift`+`Tab`	モード切り替え（plan mode など）

Node.js

コマンド	すること
`node -v / npm -v`	バージョンを確認
`node ファイル.js`	JavaScript のファイルを実行
`npx serve .`	いまのフォルダを `localhost:3000` で表示

付録 B　エラー逆引き

表示（の一部）	原因と対処	本文
`command not found`	そのコマンドが入っていない／PATH が通っていない。ターミナルを開き直す	3 章
`'irm' is not recognized`	PowerShell ではなく CMD で打っている	3 章
`Permission denied (publickey)`	SSH 鍵が GitHub に未登録。`ssh -T git@github.com` で確認	3・5 章
`Please tell me who you are`	`user.name` ／ `user.email` が未設定	3 章
`not a git repository`	Git の管理外のフォルダにいる。`pwd` で確認	5 章
`nothing to commit`	変更がない。保存し忘れていないか	5 章
`! [rejected] (fetch first)`	GitHub 側が進んでいる。`git pull` してから push	5 章
このシステムではスクリプトの実行が無効	PowerShell の安全機能。npm は Git Bash で打つ	3 章
`X is not defined`	変数名の打ち間違い（大文字・小文字も）	8 章
`Cannot read properties of null`	`querySelector` で要素が見つからない	8 章
`Cannot read properties of undefined`	配列の範囲外や、存在しない名前を読んだ	8 章
`Unexpected token`	かっこやカンマの過不足	8 章
（画像が表示されない）	パスや大文字・小文字の違い。開発者ツールの Network タブで赤い行を探す	7 章
（GitHub Pages が 404）	`index.html` の名前と置き場所、Actions タブを確認	7 章

付録 C　用語集

API キー外部サービスを使うための合言葉。秘密情報なので Git に入れない

CLI ／ GUI文字で操作する画面／マウスで操作する画面

CSSWeb ページの見た目を決める言葉

DOMブラウザが HTML を読み込んで作る「木」の形のデータ。JavaScript はこれを操作する

FlexboxCSS で要素を横（または縦）に並べる仕組み

GitHub Pages GitHub のリポジトリを、そのまま Web サイトとして公開する機能

HTMLWeb ページの中身と構造を書く言葉

localhost自分の PC を指す名前

Markdown`#` で見出し、`-` で箇条書き、のように記号で書式を付ける書き方

Node.jsブラウザの外で JavaScript を動かす道具

npm ／ npx Node.js の道具（パッケージ）を入れる／その場で実行する仕組み

PATHコマンドを探しに行くフォルダの一覧

SSH 鍵パスワードの代わりに本人確認をするための鍵の組（秘密鍵と公開鍵）

コミットGit の記録の単位。「セーブ」に近い

コンソール開発者ツールの中の、JavaScript を打ち込める場所。エラーもここに出る

差分（diff）変更前と変更後の違い

作業場（ワーキングツリー）いま編集しているファイルがある場所

準備エリア（ステージ）次のコミットに入れる予定の変更を置く場所

セレクタCSS で「どの要素に」効かせるかを指定する部分

ターミナル文字でコンピュータに命令する画面。この教材では Git Bash

デプロイ作ったものを、インターネットで見られる状態にすること

ブランチ履歴の枝分かれ。第 2 巻で使う

フレームワークWeb アプリを作るための土台一式。Next.js など

プロンプト(1) ターミナルの `$` の部分 (2) AI への指示文

ポート番号PC の中の「窓口の番号」。`localhost:3000` の `3000`

メディアクエリ画面の幅などの条件で、CSS を切り替える書き方

リポジトリGit で管理しているフォルダ（とその履歴）

リモート／ origin GitHub 側のリポジトリ／そのあだ名

レスポンシブ画面の大きさに合わせてレイアウトが変わること

付録 D　練習問題の解答例

解答は一例です。違う書き方でも、条件を満たしていれば正解です。自分の答えと違ったら、「どちらが読みやすいか」「なぜその書き方なのか」を考えてみてください。

第 7 章

練習 7.1　見出しの階層

```
<h1>理工体剣道部の一年</h1>
<h2>春</h2>
<h3>新歓</h3>
<p>新入生の体験稽古を行う。</p>
<h3>早慶戦</h3>
<p>伝統の一戦。</p>
<h2>夏</h2>
<h3>夏合宿</h3>
<p>4 泊 5 日で鍛える。</p>
```

見た目の大きさではなく、**話の階層**で見出しの番号を選びます。大きさは CSS で変えられます。

練習 7.2　外部リンク

```
<p>部の Instagram は
<a href="https://www.instagram.com/" target="_blank" rel="noopener noreferrer">こちら</a></p>
```

`target="_blank"` で新しいタブが開きます。`rel="noopener noreferrer"` は、開いた先のページから元のページを操作されないようにする安全のための設定です。

練習 7.3　画像と代わりの文章

```
<img src="images/hero.jpg" alt="稽古の後に体育館で撮った部員の集合写真" width="300">
```

`alt` は、画像が表示できないときや読み上げソフトで読まれる文章です。「画像」「写真」とだけ書くのではなく、何が写っているかを書きます。

練習 7.4　持ち物リスト

```
<h3>体験稽古の持ち物</h3>
<ul>
  <li>動きやすい服装</li>
  <li>タオル</li>
  <li>飲み物</li>
  <li>上履き（体育館用）</li>
</ul>
<h3>入部までの流れ</h3>
<ol>
  <li>見学する</li>
  <li>体験稽古に出る</li>
  <li>入部届を出す</li>
</ol>
```

順番に意味がないものは `ul`、順番に意味があるものは `ol` を使います。

練習 7.5　年間行事の表

```
<table>
  <thead>
    <tr><th>月</th><th>行事</th><th>区分</th></tr>
  </thead>
  <tbody>
    <tr><td>4 月</td><td>新歓・体験稽古</td><td>イベント</td></tr>
    <tr><td>6 月</td><td>早慶戦</td><td>試合</td></tr>
    <tr><td>8 月</td><td>夏合宿</td><td>合宿</td></tr>
    <tr><td>3 月</td><td>春合宿</td><td>合宿</td></tr>
  </tbody>
</table>
```

練習 7.6　間違い探し

1. `<h2>稽古について<h2>`：終了タグの `/` がない → `</h2>`
2. `<strong>週 4 回</p></strong>`：閉じる順番が逆。後に開いたものから先に閉じる → `<strong>週 4 回</strong></p>`
3. `<ul>` の中に `<p>`：`ul` の直下には `li` しか置けない → `<li>火曜日</li>`
4. `<img>` に `alt` がない → `alt="..."` を付ける

ブラウザは間違った HTML でもなんとか表示しようとするので、見た目では気づきにくいのが厄介なところです。

練習 7.7　問い合わせフォーム

```
<form>
  <label for="name">名前</label>
  <input type="text" id="name" name="name" required>

  <label for="email">メールアドレス</label>
  <input type="email" id="email" name="email" required>

  <label for="message">問い合わせ内容</label>
  <textarea id="message" name="message" rows="5"></textarea>

  <button type="submit">送信する</button>
</form>
```

`label` の `for` と `input` の `id` を同じ値にすると、ラベルを押したときに入力欄が選ばれます。`type="email"` でメールの形のチェック、`required` で空欄のチェックをブラウザがしてくれます。ただし、これは**ブラウザ側のチェックにすぎず、悪意のある人は簡単にすり抜けられます**。第４巻で、サーバー側でもチェックする理由を学びます。

練習 7.8　サイトの色を一気に変える

```
:root {
  --color-kon: #1e4d3a;
  --color-shu: #b8860b;
  --color-bg: #f6f7f9;
  --color-text: #2b2b2b;
}
```

CSS 変数を使うと、**色を変えたいときに直す場所が 1 か所で済み、直し忘れが起きません**。

練習 7.9　セレクタはどれに効く？

赤くなるのは「新歓稽古を行います。」と「雨天でも実施します。」の２つです。

- `.news p`：`.news` の中の `p` すべて。`class="important"` が付いていても `p` なので当てはまる
- `#top`：`id="top"` の要素はないので、どこにも効かない
- `h3`：`h3` はないので、どこにも効かない
- `section` の外の `p.important` は、`.news` の中ではないので赤くならない

練習 7.10　箱の幅を計算する

1. 指定なし：中身 300 ＋ 左右の padding 20×2 ＋ 左右の枠線 2×2 ＝ **344px**
2. `border-box`：padding と枠線が 300px の中に含まれるので **300px**

どちらの場合も、margin（10px）は箱の外側の余白なので、横幅には含みません（となりとの間隔になります）。開発者ツールの **Computed** タブに、この箱の図と数字が表示されます。

練習 7.11　ボタンに反応をつける

```
.button {
  /* もとの指定はそのまま。この 1 行を足す */
  transition: background-color 0.2s;
}

.button:hover {
  background-color: #8f3424;
}
```

練習 7.12　フッターを 3 列にする

```
<footer class="site-footer">
  <div class="footer-inner">
    <div>
      <p class="footer-title">理工体剣道部</p>
      <p>慶應義塾大学 理工学部體育會剣道部</p>
    </div>
    <div>
      <p class="footer-title">稽古場所</p>
      <p>矢上キャンパス 体育館</p>
    </div>
    <div>
      <p class="footer-title">SNS</p>
      <p><a href="https://www.instagram.com/">Instagram</a></p>
    </div>
  </div>
</footer>
.footer-inner {
  display: flex;
  justify-content: space-between;
  max-width: 960px;
```

```
  margin: 0 auto;
  text-align: left;
}

.footer-title {
  font-weight: bold;
}

.site-footer a {
  color: white;
}

@media (max-width: 640px) {
  .footer-inner {
    flex-direction: column;
  }
}
```

練習 7.13　どちらの CSS が勝つ？

緑になります。同じ要素に複数のルールが当たったとき、より「具体的な」セレクタが勝ちます。これを**詳細度**といいます。おおまかには「id の数 → class の数 → タグの数」の順に比べます。

セレクタ	id	class	タグ
`#main-title`	1	0	0
`h2.title`	0	1	1
`.title`	0	1	0

`.title` を勝たせるには、それ以上に具体的にします。たとえば `#main-title.title { color: red; }` は id 1・class 1 なので勝ちます。ただし、詳細度の競争を始めるとどんどん読みにくくなるので、**そもそも id で見た目を付けない**のが一番の対策です。

練習 7.14　タブレットでは 2 列にする

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.card {
  /* 3 枚並べる：全体から隙間 2 つ分を引いて 3 等分 */
  flex: 0 0 calc((100% - 32px) / 3);
```

```
}

@media (max-width: 960px) {
  .card {
    flex-basis: calc((100% - 16px) / 2);
  }
}

@media (max-width: 640px) {
  .card {
    flex-basis: 100%;
  }
}
```

もとの `@media (max-width: 640px)` の中の `.cards { flex-direction: column; }` は、消しても同じ見た目になります。

練習 7.15　開発者ツールで原因を探す

1. **Elements** で `h2` を選ぶと、右の **Styles** に `.section h2` のルールが出ます。右上に `style.css:行番号` と表示され、そこに `border-left: 6px solid var(--color-shu);` があります
2. `6px` をクリックして `12px` に変えると、線が太くなります
3. 再読み込みすると 6px に戻ります。開発者ツールでの変更は、ファイルには保存されません

練習 7.16　見本を再現する

```
<section class="section">
  <h2>数字で見る剣道部</h2>
  <div class="stats">
    <div class="stat"><p class="stat-number">52</p><p class="stat-label">部員数
</p></div>
    <div class="stat"><p class="stat-number">4</p><p class="stat-label">週の稽古
回数</p></div>
    <div class="stat"><p class="stat-number">2</p><p class="stat-label">年間の合宿
</p></div>
    <div class="stat"><p class="stat-number">70</p><p class="stat-label">初心者・
リバ剣の割合（%）</p></div>
  </div>
</section>
```

```
.stats {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 12px;
}

.stat {
  padding: 16px;
  text-align: center;
  background-color: white;
  border-radius: 8px;
}

.stat-number {
  margin: 0;
  font-size: 40px;
  font-weight: bold;
  color: var(--color-shu);
}

.stat-label {
  margin: 0;
  font-size: 12px;
  color: #888;
}

@media (max-width: 640px) {
  .stats {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

第 8 章

練習 8.1　稽古の合計時間

```
const minutesPerPractice = 120;
const practicesPerWeek = 4;
const weeks = 15;

const total = minutesPerPractice * practicesPerWeek * weeks;
console.log(`前期の稽古時間は ${total} 分`);   // → 7200 分
```

数字に名前をつけておくと、あとで読んだときに「120 って何だっけ」とならずに済みます。

練習 8.2　型を当てる

```
typeof "4"          // "string"
typeof 4            // "number"
"4" + 4             // "44"   文字列があると、+ は「つなげる」になる
4 + 4               // 8
"4" === 4           // false  型が違う
Number("4") === 4   // true
```

練習 8.3　技の配列

```
const waza = ["面", "小手", "胴"];

waza.push("突き");
console.log(waza, waza.length);          // → ["面","小手","胴","突き"] 4

console.log(waza[1]);                    // → 小手

console.log(waza[waza.length - 1]);      // → 突き
```

`const` で作った配列でも、`push` で中身を足すことはできます。`const` が禁止しているのは「箱ごと入れ替えること」だけです。

練習 8.4　自分のプロフィール

```
const me = {
  name: "柴山",
  grade: 2,
  department: "応用化学科",
  favoriteWaza: "出小手",
};

console.log(`${me.department}${me.grade}年の${me.name}です。好きな技は
${me.favoriteWaza}です。`);
```

練習 8.5　分を「時間と分」に直す関数

```
function formatMinutes(totalMinutes) {
  const hours = Math.floor(totalMinutes / 60);
  const minutes = totalMinutes % 60;
  return `${hours}時間${minutes}分`;
}
```

```
// 0 の部分を省略する版
function formatMinutesShort(totalMinutes) {
  const hours = Math.floor(totalMinutes / 60);
  const minutes = totalMinutes % 60;
  if (hours === 0) return `${minutes}分`;
  if (minutes === 0) return `${hours}時間`;
  return `${hours}時間${minutes}分`;
}

console.log(formatMinutes(150));        // → 2 時間 30 分
console.log(formatMinutesShort(60));    // → 1 時間
console.log(formatMinutesShort(45));    // → 45 分
```

練習 8.6　上級生と下級生

```
function gradeLabel(grade) {
  if (grade === 1) {
    return "新入生";
  } else if (grade === 2) {
    return "中堅";
  } else if (grade >= 3) {
    return "上級生";
  } else {
    return "学年が正しくありません";
  }
}

[1, 2, 3, 4, 0].forEach((g) => console.log(g, gradeLabel(g)));
```

練習 8.7　段位を持っている人

```
const withDan = members.filter((member) => member.dan !== "無");
console.log(withDan.length);
withDan.forEach((member) => console.log(member.name));
```

練習 8.8　さん付けの名簿

```
const roster = members.map((member) => `${member.name}さん（${member.grade}年）`);
console.log(roster);
console.log(roster.join("\n"));
```

`\n` は改行を表す特別な文字です。

練習 8.9　学年ごとの人数

```
const counts = {};

members.forEach((member) => {
  // まだその学年の数を数えていなければ 0 から始める
  if (counts[member.grade] === undefined) {
    counts[member.grade] = 0;
  }
  counts[member.grade] = counts[member.grade] + 1;
});

console.log(counts);   // → { '1': 2, '2': 2, '3': 2 }
```

オブジェクトの名前（キー）を、`counts[member.grade]` のように変数で指定しているのがポイントです。この「グループごとに数える」処理は、第 3 巻の SQL の `GROUP BY` でもう一度出てきます。

練習 8.10　素振りカウンター

```
<p class="count">0</p>
<button class="plus">素振り +1</button>
<button class="reset">リセット</button>

<style>
  .count.is-goal { color: #b5422e; font-weight: bold; }
</style>

<script>
  const GOAL = 100;
  const countText = document.querySelector(".count");
  let count = 0;

  function update() {
    countText.textContent = count;
    if (count >= GOAL) {
      countText.classList.add("is-goal");
    } else {
      countText.classList.remove("is-goal");
    }
  }

  document.querySelector(".plus").addEventListener("click", () => {
    count = count + 1;
```

```
    update();
  });

  document.querySelector(".reset").addEventListener("click", () => {
    count = 0;
    update();
  });
</script>
```

`count` はあとで値を入れ直すので `let`、目標の 100 は変えないので `const` です。

練習 8.11　名前で検索する

```
<!-- index.html：部員紹介の .filter の下に足す -->
<input type="text" class="member-search" placeholder="名前で検索">
// script.js：一番最後に足す
const searchInput = document.querySelector(".member-search");

searchInput.addEventListener("input", () => {
  const keyword = searchInput.value.trim();
  const shown = members.filter((member) => member.name.includes(keyword));

  memberList.innerHTML = shown
    .map((member) => `<li \
+     ↪ class="member"><strong>${member.name}</strong><br>${member.grade}年・
${member.dan}</li>`)
    .join("");
  memberCount.textContent = `${shown.length}人`;
});
```

この解答では、学年のボタンと検索欄が別々に動きます（検索すると学年の絞り込みが無視される）。両方を同時に効かせる方法は、本章の腕だめしで考えます。

練習 8.12　バグを直す

```
const list = document.querySelector(".member-list");   // ① lsit → list

const grade = 2;                                        // ② 文字列 "2" ではなく
数値 2

const second = members.filter((member) => member.grade === grade);

list.innerHTML = second.map((member) => `<li>${member.name}</li>`).join("");   //
③ nmae → name
```

1. クラス名の打ち間違いで要素が見つからず、`list` が `null` になっていた（Cannot

`read properties of null`）

2. `member.grade` は数値なので、文字列の `"2"` とは `===` で一致せず、誰も残らなかった（エラーは出ないので一番気づきにくい）
3. 存在しない `nmae` を読んだため、名前が `undefined` と表示されていた

②のように、**エラーが出ないのに結果がおかしい**バグが一番やっかいです。「何人残ったか」を `console.log(second.length)` で確かめるなど、途中の値を表示して追いかけるのが基本です。